RAJALAKSHMI ENGINEERING COLLEGE

AN AUTONOMOUS INSTITUTION

Affiliated to ANNA UNIVERSITY

Rajalakshmi Nagar,Thandalam,

Chennai-602105

DEPARTMENT OF COMPUTER SCIENCE

AND ENGINEERING

CS23A34 - USER INTERFACE DESIGN LABORATORY

ACADEMIC YEAR:2024-2025 (EVEN)

INDEX

Reg. No : 230701400 **Name : Niranjana.K**

Branch : CSE **Year/Section : II/D**

LIST OF EXPERIMENTS

Experiment. No.	**Title**	**Tools**
1	Design a UI where users recall visual elements (e.g., icons or text chunks). Evaluate the effect of chunking on user memory.	Figma
2	Develop and compare CLI, GUI, and Voice User Interfaces (VUI) for the same task and assess user satisfaction.	Python (Tkinter for GUI, Speech Recognition for VUI) / Terminal
3A	Create a prototype with familiar and unfamiliar navigation elements. Evaluate ease of use with different user groups.	Proto.io
3B	Create a prototype with familiar and unfamiliar navigation elements. Evaluate ease of use with different user groups.	Wireflow
4A	Conduct task analysis for an app (e.g., online shopping) and document user flows. Create corresponding wireframes.	Lucidchart (free tier)
4B	Conduct task analysis for an app (e.g., online shopping) and document user flows. Create corresponding wireframes.	Dia (open source)
5A	Simulate the lifecycle stages for UI design using the RAD model and develop a small interactive interface.	Axure RP

5B	Simulate the lifecycle stages for UI design using the RAD model and develop a small interactive interface.	OpenProj
6	Experiment with different layouts and color schemes for an app. Collect user feedback on aesthetics and usability.	GIMP (open source for graphics)
7A	Develop low-fidelity paper prototypes for a banking app and convert them into digital wireframes.	Pencil Project
7B	Develop low-fidelity paper prototypes for a banking app and convert them into digital wireframes.	Inkscape
8A	Create storyboards to represent the user flow for a mobile app (e.g., food delivery app).	Balsamiq
8B	Create storyboards to represent the user flow for a mobile app (e.g., food delivery app).	OpenBoard
9	Design input forms that validate data (e.g., email, phone number) and display error messages.	HTML/CSS, JavaScript (with Validator.js)
10	Create a data visualization (e.g., pie charts, bar graphs) for an inventory management system.	JavaScript

EXERCISE 1: Memory recall UI

Aim:

To design a UI where users recall visual elements (e.g., icons or text chunks) and evaluate the effect of chunking on user memory.

Procedure:

A. Home Screen:

1. **Create a Frame** (1024x768px for desktop).
2. **Add Instructions**: Use text for headings and detailed instructions.
3. **Start Button**: Create a button with text "Start" and link it to the next screen (Chunking Phase).

B. Chunking Phase:

1. **Create a New Frame** for the Chunking Phase.
2. **Design Chunked Items**: Group 3-5 items (icons/text) into chunks (with or without borders).
3. **Set Viewing Time**: Simulate time by setting a 5-second transition to the next screen.

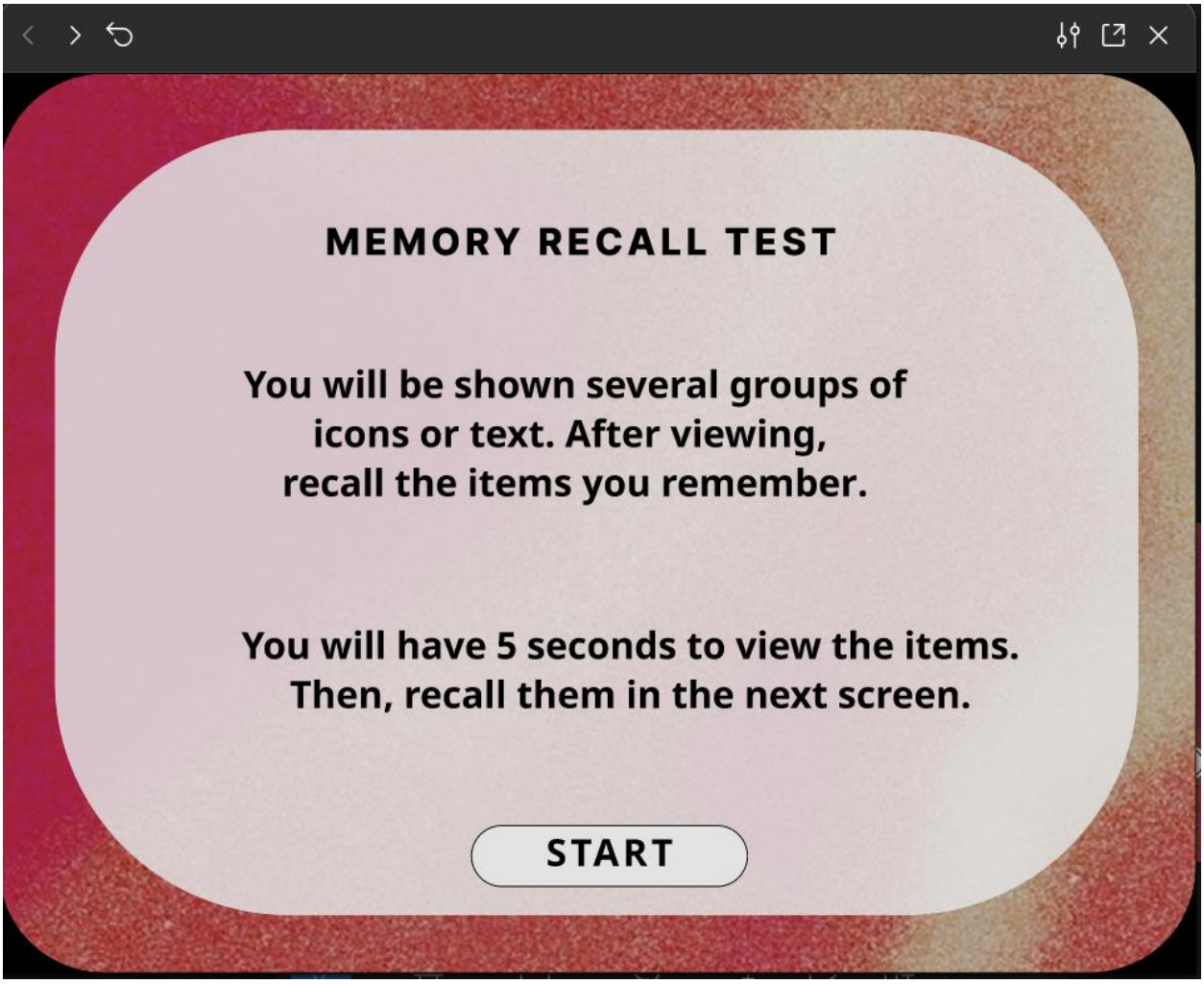
C. Recall Phase:

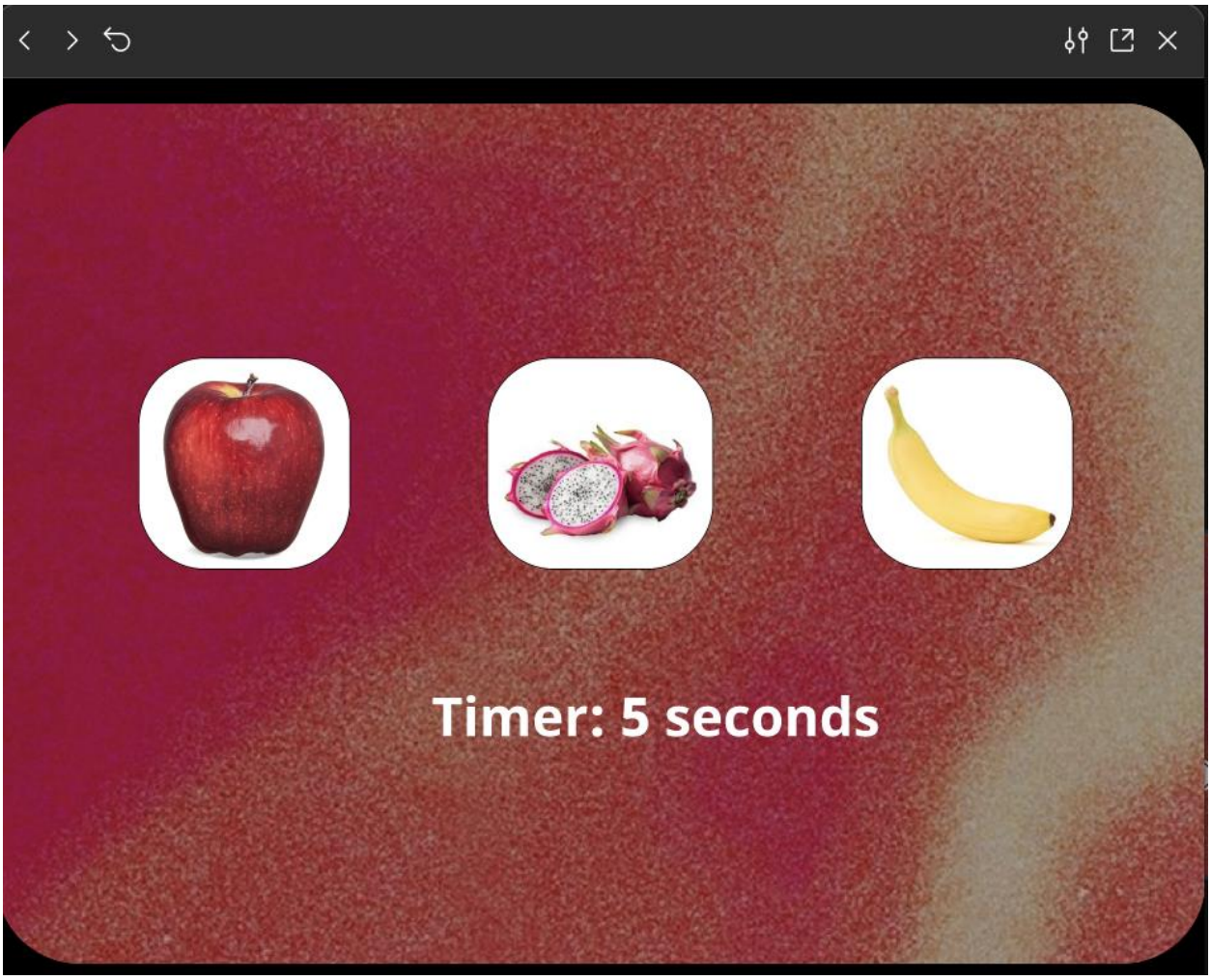
1. **Create a New Frame** for recall.
2. **Recall Input**: Use either multiple-choice (checkboxes/radio buttons) or text input fields for users to recall items.
3. **Submit Button**: Create a "Submit Recall" button and link it to the next screen (Feedback).

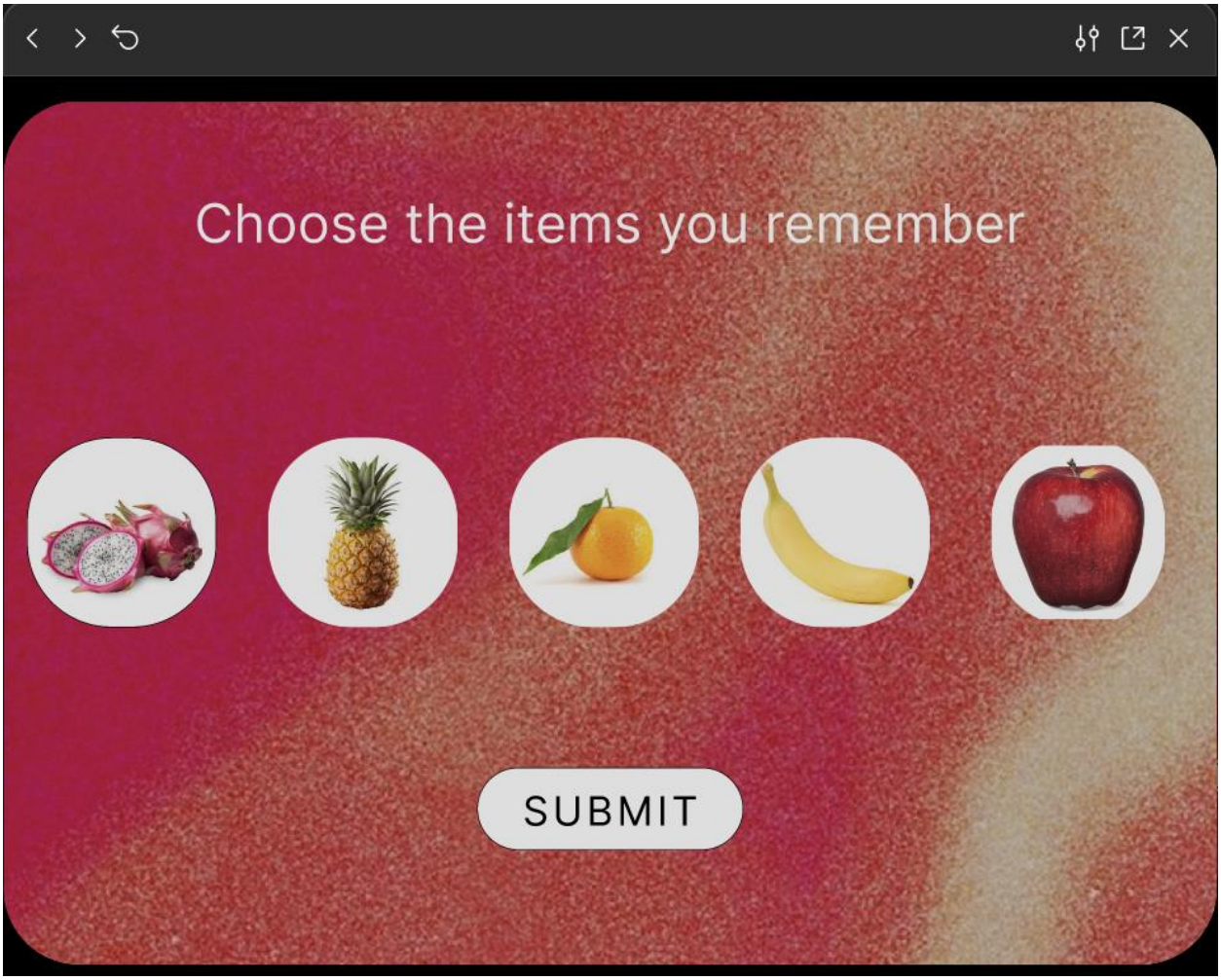
D. Result Screen:

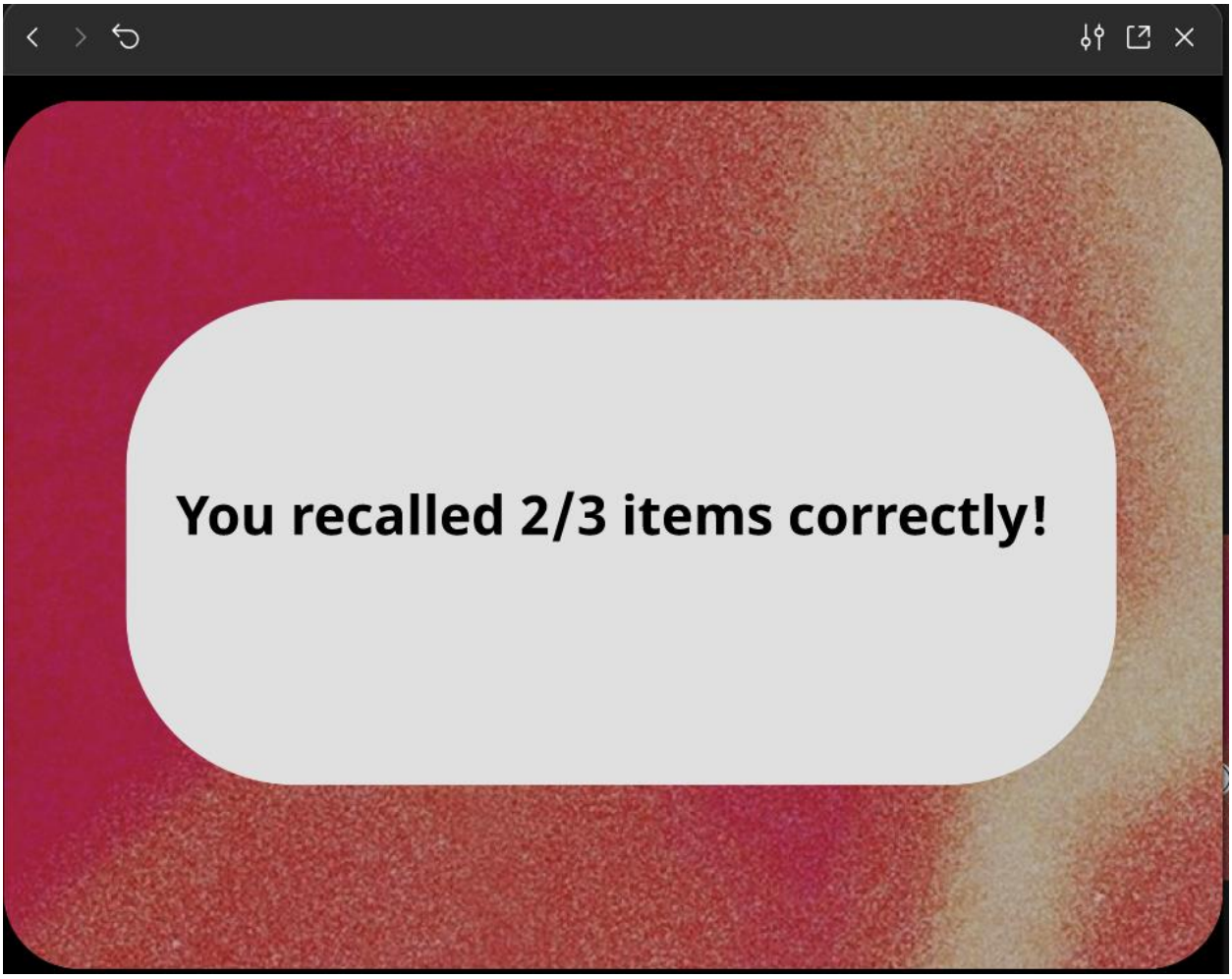
1. **Feedback Screen**: Display recall accuracy (e.g., “You recalled 4/5 items correctly!”).
2. **Analyze**: Vary chunk size (3 vs. 5 items) and chunk type (icons vs. text) for testing.

Output:


MEMORY RECALL TEST
You will be shown several groups of icons or text. After viewing, recall the items you remember.
You will have 5 seconds to view the items. Then, recall them in the next screen.
START


Timer: 5 seconds


Choose the items you remember
SUBMIT



LINK:

https://www.figma.com/proto/8tQTaum1Ose8M70q5FYkoV/Untitled?node-id=1-2&t=SZnpx7mkYL9AatVN-1&scaling=min-zoom&content-scaling=fixed&page-id=0%3A1&starting-point-node-id=1%3A2

Result:

The **Memory Recall UI** successfully tests chunking effects by displaying grouped icons/text, prompting recall, and providing feedback on user memory accuracy.

EXERCISE 2: CLI, GUI AND VUI

Aim:

To develop and compare Command Line Interface (CLI), Graphical User Interface (GUI), and Voice User Interface (VUI) for the same task, and assess user satisfaction using Python (with Tkinter for GUI and Speech Recognition for VUI) and Terminal.

Procedure:

1. **CLI (Command-Line Interface)**

- **Set Up**:
  - Install Python and create a new script (e.g., todo.py).
- **Core Functions**:
  - add_task(task): Adds a task.
  - view_tasks(): Displays all tasks with numbering.
  - remove_task(task_number): Deletes a task by index.
  - save_tasks(): Saves tasks to a file.
  - load_tasks(): Loads tasks from a file at startup.
- **User Interaction**:
  - Use input() to capture user commands (add, view, remove, exit).
  - Implement a loop to continuously handle commands.
- **File Storage**:
  - Tasks are saved in tasks.txt.
  - Use file handling functions (open()) for reading and writing tasks.

- **Test & Run**:
    - Test the functions by running the script and checking task management functionality.

2. GUI (Graphical User Interface)

- **Set Up**:
    - Install Python, Tkinter (or PyQt6).
- **Create GUI Layout**:
    - Main window, entry box for task input, listbox for task display, buttons for add, remove, and clear actions.
- **Task Management Functions**:
    - add_task(): Adds a task from the input box to the list.
    - remove_task(): Deletes a selected task.
    - clear_tasks(): Clears all tasks.
    - save_tasks(): Saves tasks in a file.
    - load_tasks(): Loads tasks at startup.
- **Event Handling**:
    - Bind buttons to respective functions and implement task removal on double-click.
- **Test & Run**:
    - Ensure all actions (add, remove, clear) work correctly in the GUI and run the script.

3. VUI (Voice User Interface)

- **Install Dependencies**:
    - Install necessary libraries: speechrecognition, pyttsx3, pyaudio.
- **Set Up Speech Recognition & Synthesis**:

- Use speech_recognition to convert speech to text.
- Use pyttsx3 for text-to-speech responses.

- **Voice-Controlled Functions**:
  - listen_command(): Captures voice input.
  - add_task(task): Adds a task based on spoken input.
  - remove_task(task_number): Removes a task using voice input.
  - speak(text): Provides audio feedback.
- **Command Processing**:
  - Recognize voice commands (e.g., "Add task: Buy groceries" or "Remove task 2") and map them to functions.
- **Test & Improve**:
  - Test the script with different commands and refine speech recognition accuracy.

Program & Output:

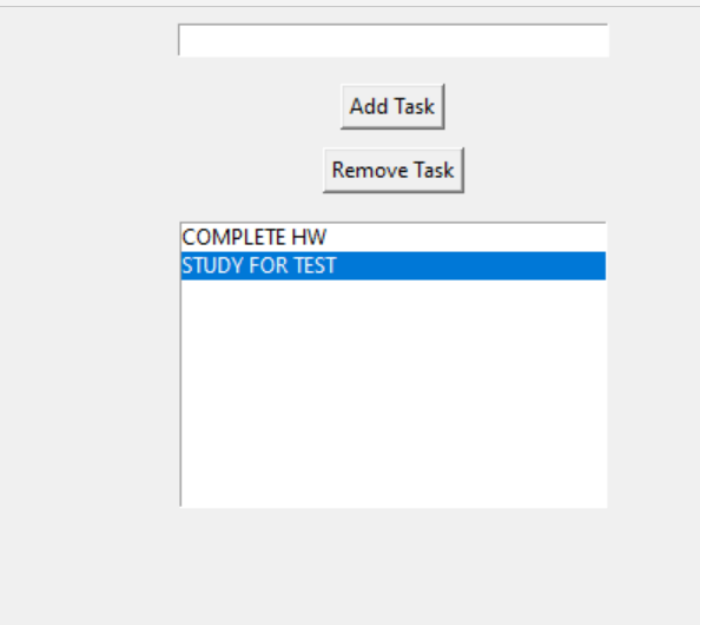
CLI:

```
tasks=[]

def add_task(task):
    tasks.append(task)
    print(f"Task '{task}'added.")
def view_tasks():
    if tasks:
        print("Your tasks:")
        for idx,task in enumerate(tasks, 1):
            print(f"{idx}.{task}")
    else:
        print("no tasks to show")
def remove_task(task_number):
    if 0<task_number<=len(tasks):
        removed_task=tasks.pop(task_number-1)
        print(f"Task'{removed_task}' removed")
    else:
        print("Invalid task number.")
def main():
    while True:
        print("\nOptions: 1.add task 2.view task 3.remove task 4.exit")
        choice=input("enter your choice")
        if choice=='1':
            task=input("enter task:")
            add_task(task)
        elif choice=='2':
            view_tasks()
        elif choice=='3':
            task_number=int(input("enter task number to remove:"))
            remove_task(task_number)
        elif choice=='4':
            print("exiting")
            break
        else:
            print("invalid choice.try again.")
if __name__=="__main__":
    main()
```

```
Options: 1.add task 2.view task 3.remove task 4.exit
enter your choice
invalid choice.try again.

Options: 1.add task 2.view task 3.remove task 4.exit
enter your choice1
enter task:do hw
Task 'do hw'added.

Options: 1.add task 2.view task 3.remove task 4.exit
enter your choice2
Your tasks:
1.do hw

Options: 1.add task 2.view task 3.remove task 4.exit
enter your choice3
enter task number to remove:1
Task'do hw' removed

Options: 1.add task 2.view task 3.remove task 4.exit
enter your choice2
no tasks to show

Options: 1.add task 2.view task 3.remove task 4.exit
enter your choice
```

GUI:

```
import tkinter as tk
from tkinter import messagebox
tasks = []
def add_task():
    task = task_entry.get()
    if task:
        tasks.append(task)
        task_entry.delete(0, tk.END)
        update_task_list()
    else:
        messagebox.showwarning("Warning", "Task cannot be empty")

def update_task_list():
    task_list.delete(0, tk.END)
    for task in tasks:
        task_list.insert(tk.END, task)
def remove_task():
    selected_task_index = task_list.curselection()
    if selected_task_index:
        task_list.delete(selected_task_index)
        tasks.pop(selected_task_index[0])
app = tk.Tk()
app.title("To-Do List")
task_entry = tk.Entry(app, width=40)
task_entry.pack(pady=10)
add_button = tk.Button(app, text="Add Task", command=add_task)
add_button.pack(pady=5)
remove_button = tk.Button(app, text="Remove Task", command=remove_task)
remove_button.pack(pady=5)
task_list = tk.Listbox(app, width=40, height=10)
task_list.pack(pady=10)
app.mainloop()
```



VUI:

```
import speech_recognition as sr
import pyttsx3
tasks = []
recognizer = sr.Recognizer()
engine = pyttsx3.init()
def add_task(task):
    tasks.append(task)
    engine.say(f"Task {task} added")
    engine.runAndWait()
def view_tasks():
    if tasks:
        engine.say("Your tasks are")
        for task in tasks:
            engine.say(task)
    else:
        engine.say("No tasks to show")
    engine.runAndWait()
def remove_task(task_number):
    if 0 <task_number <= len(tasks):
        removed_task = tasks.pop(task_number - 1)
        engine.say(f"Task {removed_task} removed")
    else:
        engine.say("Invalid task number")
    engine.runAndWait()
def recognize_speech():
    with sr.Microphone() as source:
        print("Listening...")
        audio = recognizer.listen(source)
        try:
            command = recognizer.recognize_google(audio)
            return command
        except sr.UnknownValueError:
            engine.say("Sorry, I did not understand that")
            engine.runAndWait()
            return None
```

```python
def main():
    while True:
        engine.say("Options: add task, view tasks, remove task, or exit")
        engine.runAndWait()
        command = recognize_speech()
        if not command:
            continue
        if "add task" in command:
            engine.say("What is the task?")
            engine.runAndWait()
            task = recognize_speech()
            if task:
                add_task(task)
        elif "view tasks" in command:

            view_tasks()
        elif "remove task" in command:
            engine.say("Which task number to remove?")
            engine.runAndWait()
            task_number = recognize_speech()
            if task_number:
                remove_task(int(task_number))
        elif "exit" in command:
            engine.say("Exiting...")
            engine.runAndWait()
            break
        else:
            engine.say("Invalid option. Please try again.")
            engine.runAndWait()
if __name__ == "__main__":
    main()
```

```
Listening...
Listening...
Listening...
Listening...
Listening...
Listening...
Listening...
Listening...
Listening...
Listening...
```

Result:

The experiment successfully developed a To-Do application with CLI, GUI, and VUI interfaces, each allowing efficient task management and seamless user interaction.

Exercise 3a: Create a prototype with familiar and unfamiliar navigation elements. Evaluate ease of use with different user groups using proto.io

Aim:

The aim is to develop a prototype incorporating both familiar and novel navigation elements and assess usability among diverse user groups using Proto.io.

Procedure:

Step 1: Sign Up and Log In

- Go to proto.io, sign up or log in.

Step 2: Create a New Project

- Click "Create New Project," name it, select device type (e.g., iPhone X), and click "Create."

Step 3: Design the Home Screen

- Add a new screen (Blank, name it "Home").
- Drag a "Header" widget, edit text to "Home Screen."
- Add a "Button" widget, change text to "Go to Profile."
- Set button interaction: Trigger = "Tap/Click," Action = "Navigate to Screen" → create "Profile" screen.

Step 4: Design the Profile Screen

- Add a "Header" widget, edit text to "Profile Screen."
- Add an "Image" widget for the profile picture.
- Add a "Text" widget for profile info (e.g., "John Doe, Software Engineer").
- Add a "Button" (Back to Home), set interaction: Trigger = "Tap/Click," Action = "Navigate to Screen" → Home.

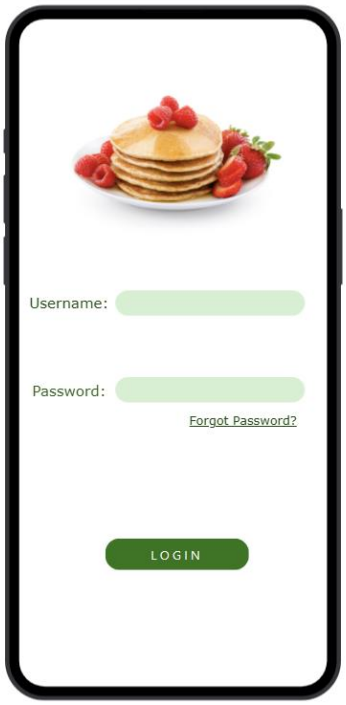
Step 5: Preview the Prototype

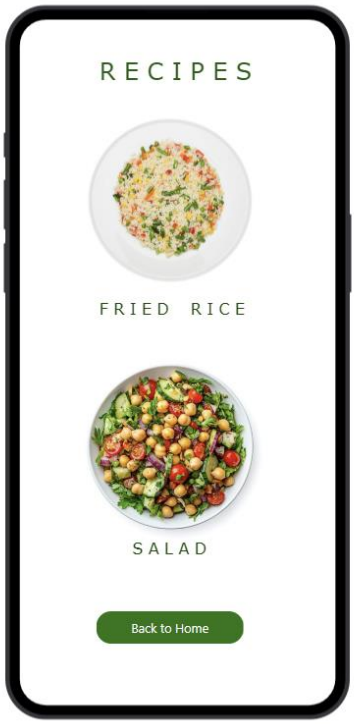
- Click "Preview" and interact with the prototype.

Step 6: Share the Prototype

- Click "Share," copy the link, and send it for feedback.

Output:





Link:

https://pr.to/NBZTTR/

Result:

Hence, the prototypes were successfully designed using Proto.io.

Exercise 3b

Create a prototype with familiar and unfamiliar navigation elements. Evaluate ease of use with different user groups using wireflow.

Aim:

The aim is to design a prototype with both well-known and new navigation elements and measure user-friendliness across different user groups using Wireflow.

Procedure:

Step 1: Plan Your Prototype

1. **Define Navigation Elements**:
    - **Familiar**: Standard menus, top bars, footers, sidebars.
    - **Unfamiliar**: Hidden menus, gesture-based navigation, custom swipes.
2. **Sketch Your Layout**:
    - Start with paper sketches or use tools like Figma/Sketch for visualizing design.

Step 2: Set Up Your Wireflow Project

1. **Sign Up/Log In**: Create an account or log in to Wireflow.
2. **Start a New Project**: Name your project and choose a template or start from scratch.

Step 3: Design the Prototype

1. **Add Familiar Navigation**: Drag and drop components like menus, buttons, etc.
2. **Incorporate Unfamiliar Elements**: Add hidden menus or gestures.
3. **Link Screens**: Use Wireflow's tools to connect screens and create transitions.

Step 4: Prepare for Usability Testing

1. **Identify User Groups**: Segment users by age, tech-savviness, or experience.
2. **Recruit Participants**: Find users through tools like UserTesting, forums, or social media.

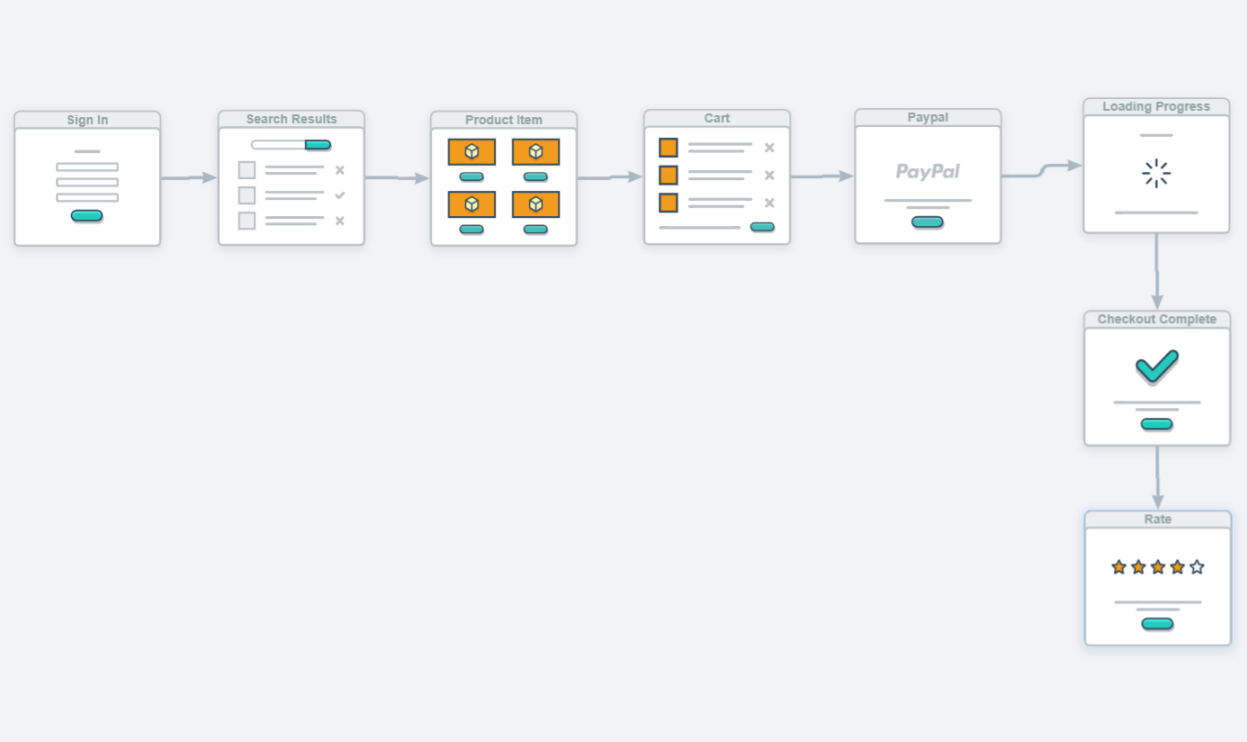
Step 5: Conduct Testing

1. **Share the Prototype**: Send a shareable link for users to interact with.
2. **Test Sessions**: Ask users to complete tasks using both familiar and unfamiliar navigation.
3. **Collect Feedback**: Use Wireflow's feedback system or conduct follow-up interviews.

Step 6: Analyze and Report

1. **Analyze Data**: Look for patterns in ease of use and preferences.
2. **Compare Results**: Examine how different user groups interacted with navigation types.
3. **Create a Report**: Summarize insights, challenges, and recommendations.

Output:



Result:

Hence, the prototype with familiar and unfamiliar navigation elements has been successfully executed using Wireflow.

EXERCISE 4a:

Conduct task analysis for an app (e.g., online shopping) and document user flows. Create corresponding wireframes using Lucidchart

Aim:

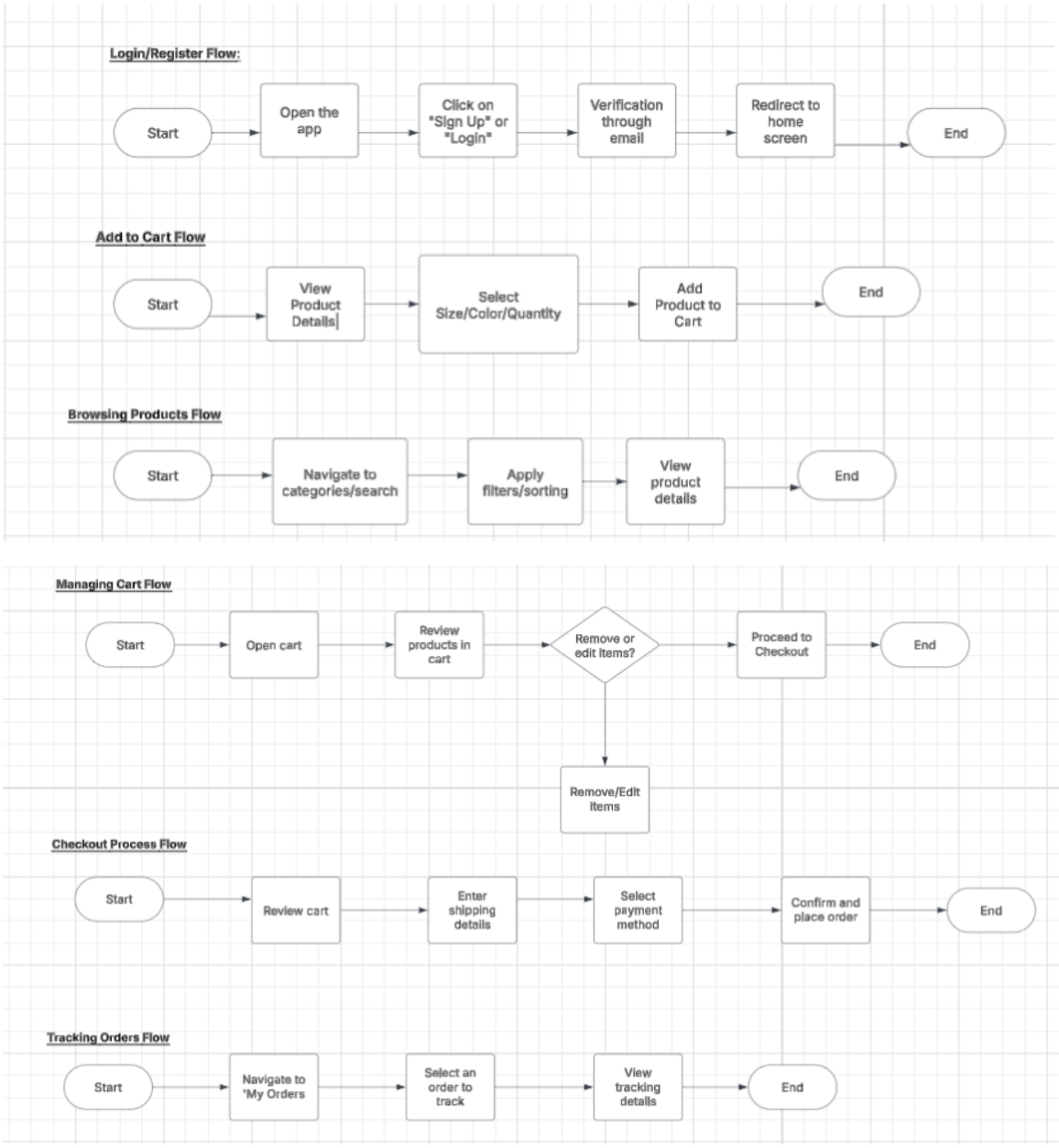
To understand and document the steps a user takes to complete the main tasks within an online shopping app.

Procedure:

1. **Create a New Document**
- Sign in to Lucidchart and click on "+ Document" or "Create New Diagram."
2. **Select a Template**
- Start with a blank document or choose a flowchart template.
3. **Add Shapes for Each Step**
- Drag shapes (rectangles for actions, diamonds for decisions) to represent steps like: Login/Register, Browsing, Adding Products, Cart Management, Checkout, and Tracking Orders.
4. **Connect the Shapes**
- Use connectors and arrows to show the flow from one step to the next.
5. **Add Details to Each Step**
- Double-click shapes to describe actions, like "Click Sign Up" or "Enter details."
6. **Use Different Shapes for Actions**
- Rectangles for actions, diamonds for decisions, and ovals for start/end points.
7. **Customize and Organize**
- Arrange shapes logically, use colors for clarity, and group related steps.
8. **Review and Save**
- Ensure everything is connected properly, then save by clicking File > Save.
9. **Share and Collaborate**

- Share the flowchart or export it as an image/PDF.

Output:



Link:

https://lucid.app/lucidchart/69926ff7-a2b7-4f1c-93c9-29ca626c5496/edit?viewport_loc=-538%2C851%2C3188%2C1287%2C0_0&invitationId=inv_a9421318-8144-46c9-bc8e-7e93f0f9edb3

Result:

Hence, the task analysis for an app has been successfully executed.

Exercise 4b

Conduct task analysis for an app (e.g., online shopping) and document user flows. Create corresponding wireframes using dia

Aim:

The aim is to perform task analysis for an app, such as online shopping, document user flows, and create corresponding wireframes using Dia.

Procedure:

1. **Install Dia:**

- Download and install Dia from dia-installer.de.
- Open the Dia application.

2. **Create New Diagram:**

- Go to File → New Diagram.
- Choose "Flowchart" as the diagram type.

3. **Add Shapes:**

- Use shape tools (rectangles, ellipses) for screens like Home Page, Product Categories, Product Listings, Cart, etc.

4. **Connect Shapes:**

- Use the line tool to connect shapes in the user flow (e.g., Home Page → Product Categories → Cart).

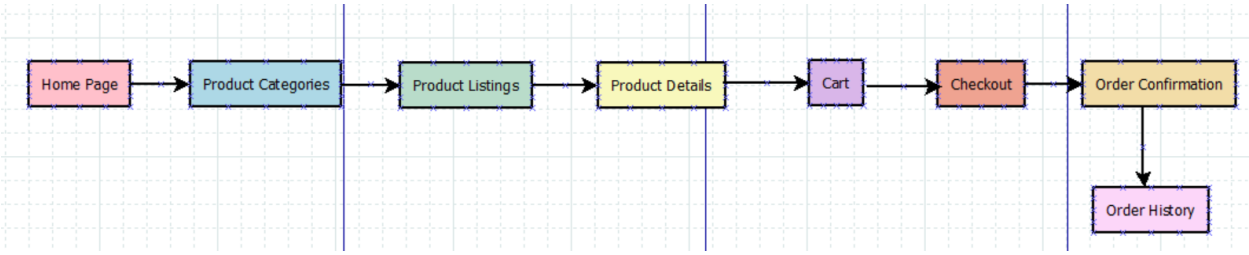
5. **Label Shapes:**

- Double-click each shape to label it (e.g., "Home Page," "Checkout").

6. **Save the Diagram:**

- Go to File → Save As and name the diagram (e.g., "Online Shopping App User Flows").

Output:



Result:

Hence, task analysis for an app has been executed successfully using Dia.

Exercise 5a:

Simulate the lifecycle stages for UI design using the RAD model and develop a small interactive interface using Axure RP

Aim:

The aim is to demonstrate the lifecycle stages of UI design via the RAD model and develop a small interactive interface employing Axure RP.

Procedure:

Phase 1: Requirements Planning

1. **Identify Key Features:**
    - Navigation (Home, Categories, Product Details, Cart, Checkout, Order Confirmation, History)
    - User Actions (Browsing, Searching, Cart Management, Checkout, Order Tracking)
2. **Create Requirements Document:**
    - List features and functionalities.
    - Document user stories and use cases.

Phase 2: User Design

1. **Install Axure RP**:
    - Download and launch Axure RP.
2. **Create New Project**:
    - Start a new project (e.g., "Shopping App Interface").
3. **Create Wireframes**:
    - Use widgets to design key screens (Home, Categories, Product Listings, Details, Cart, Checkout, Confirmation, History).

4. **Add Interactions**:
    - Define actions (e.g., OnClick) for elements (buttons, links).
5. **Create Masters**:
    - Build reusable components (e.g., header, footer).
6. **Add Annotations**:
    - Describe functionality using the Notes panel.

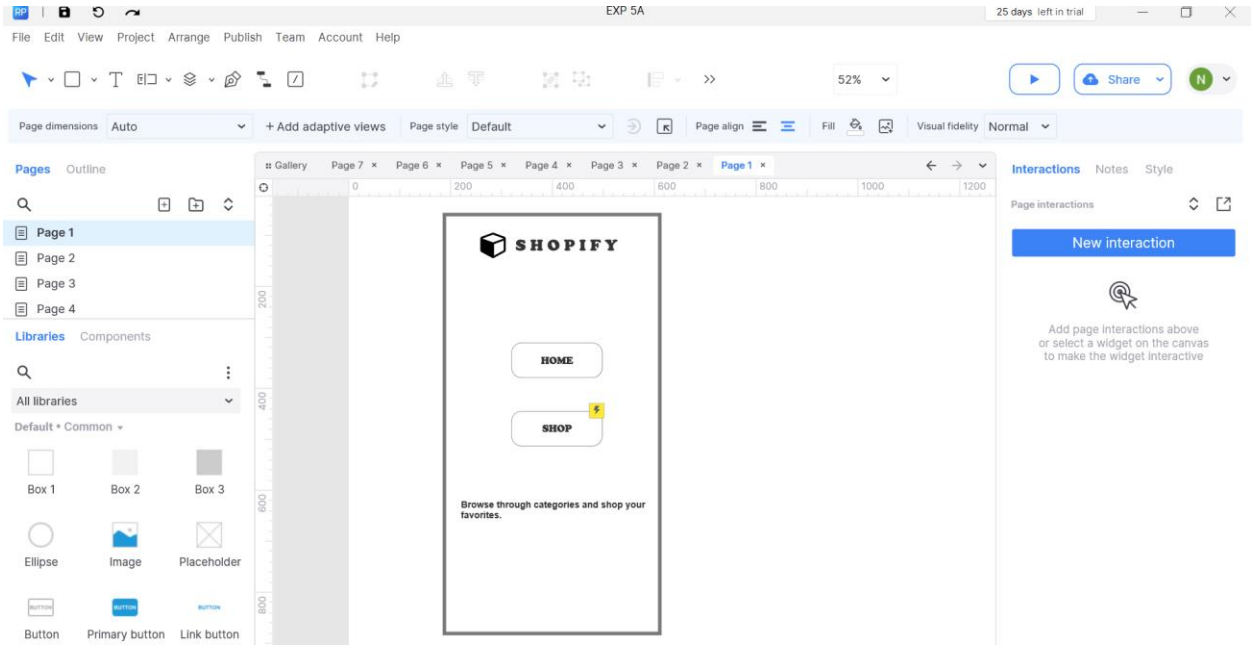
Phase 3: Construction

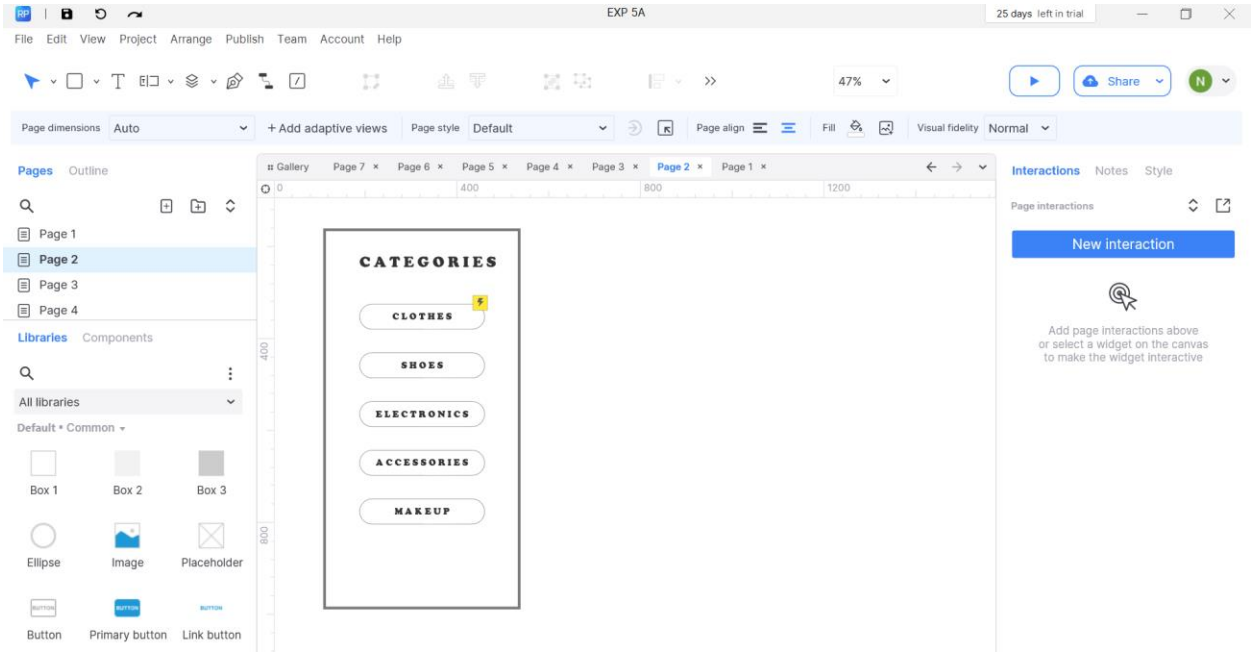
1. **Develop Interactive Prototypes**:
    - Add interactions and transitions for a dynamic experience.
    - Use dynamic panels for elements like carousels and pop-ups.
2. **Test and Iterate**:
    - Preview the prototype.
    - Gather feedback and make necessary adjustments.

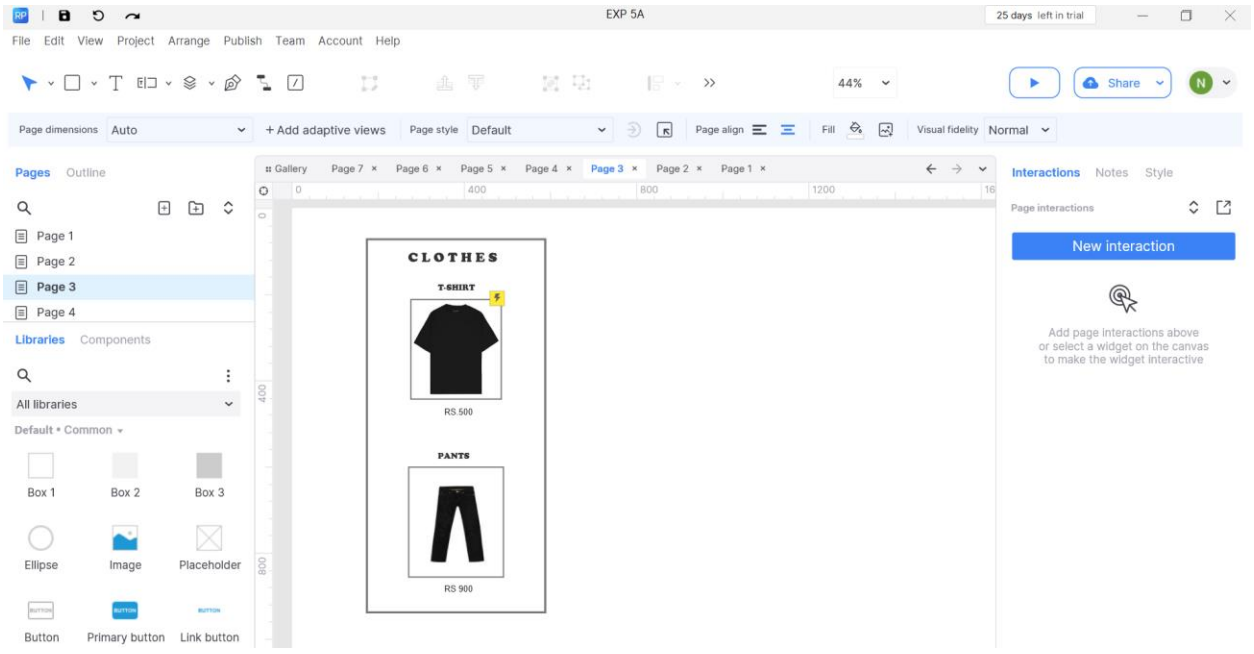
Phase 4: Cutover

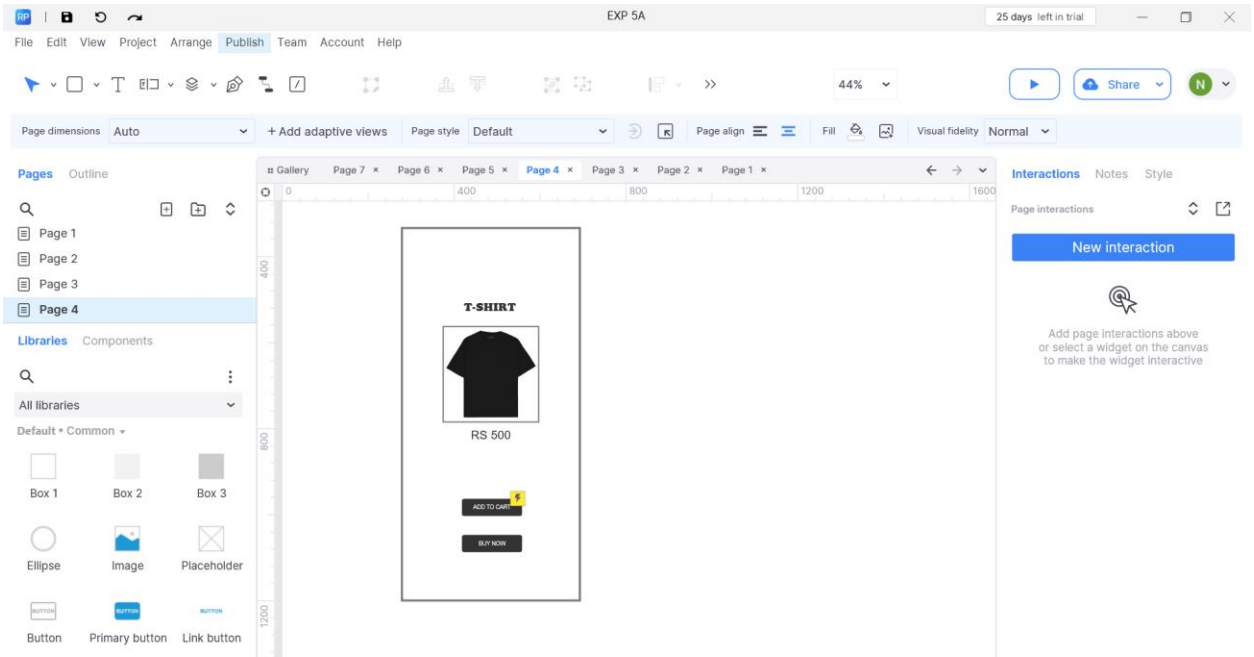
1. **Finalize and Export**:
    - Complete design and interactions.
    - Export the prototype (HTML or Axure Cloud).
2. **User Training and Support**:
    - Conduct training and provide documentation.

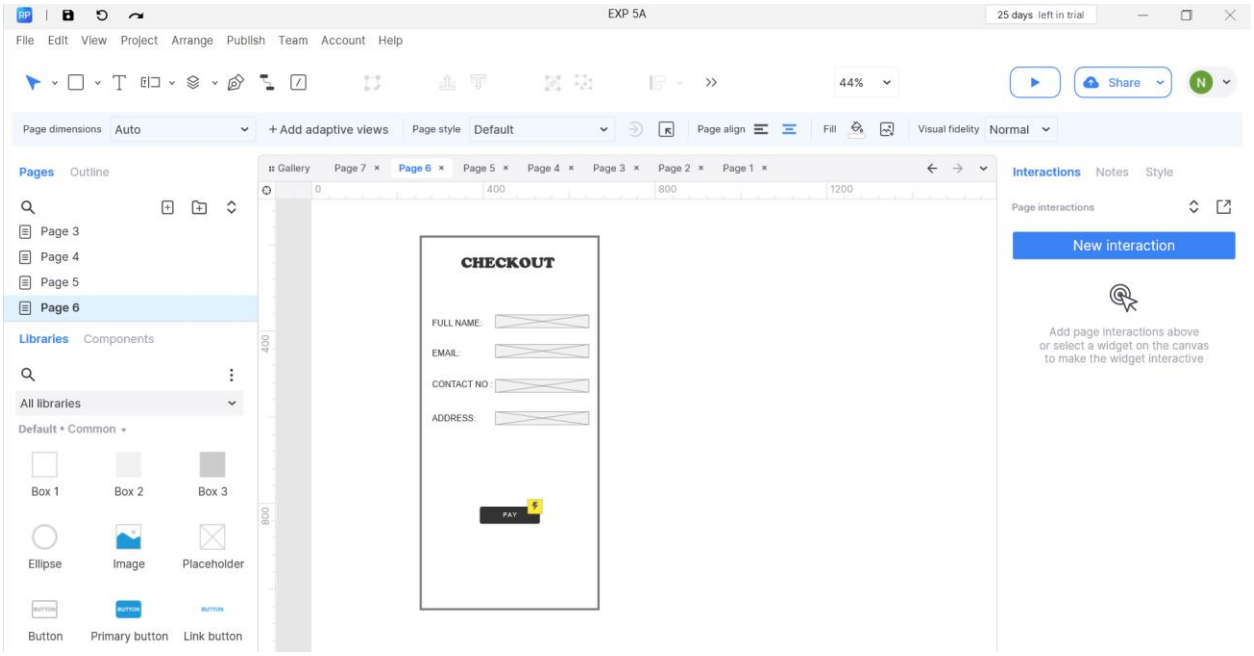
Output:

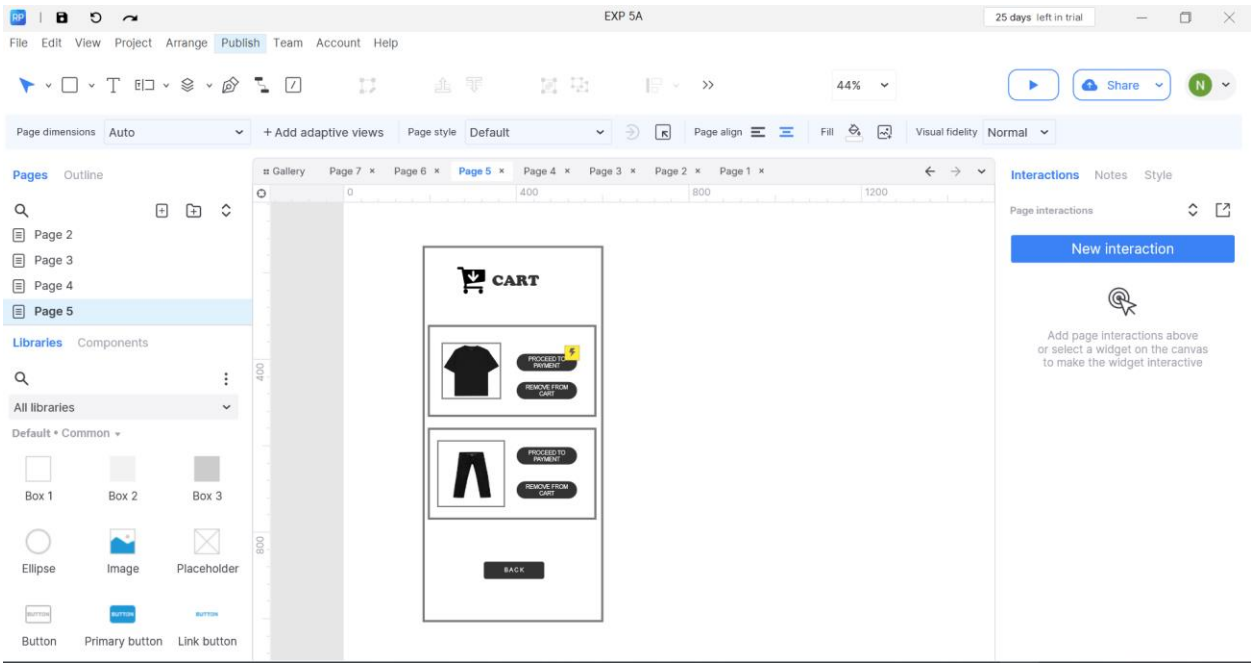

EXP 5A
25 days left in trial
File Edit View Project Arrange Publish Team Account Help
52%
Share
Page dimensions Auto
+ Add adaptive views
Page style Default
Page align
Fill
Visual fidelity Normal
Pages Outline
Page 1
Page 2
Page 3
Page 4
Libraries Components
All libraries
Default • Common
Box 1
Box 2
Box 3
Ellipse
Image
Placeholder
Button
Primary button
Link button
Gallery Page 7 Page 6 Page 5 Page 4 Page 3 Page 2 Page 1
Interactions Notes Style
Page interactions
New interaction
Add page interactions above or select a widget on the canvas to make the widget interactive
SHOPIFY
HOME
SHOP
Browse through categories and shop your favorites.

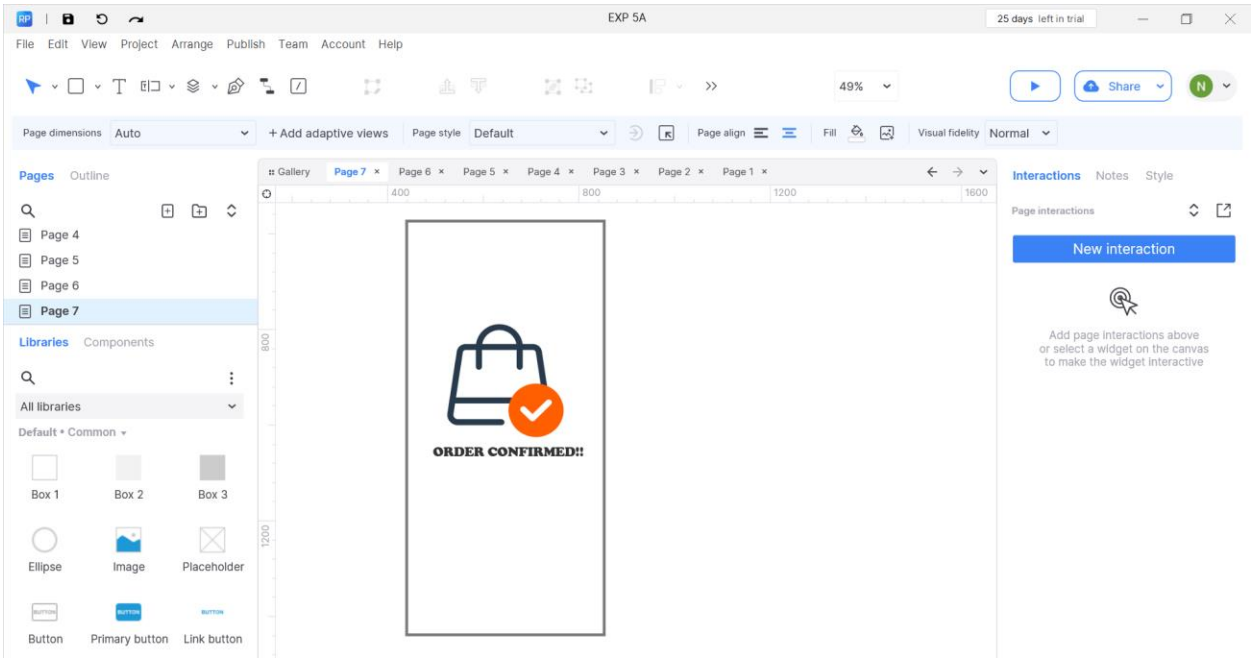

EXP 5A
25 days left in trial
File Edit View Project Arrange Publish Team Account Help
47%
Share
Page dimensions Auto
+ Add adaptive views
Page style Default
Page align
Fill
Visual fidelity Normal
Pages Outline
Page 1
Page 2
Page 3
Page 4
Libraries Components
All libraries
Default • Common
Box 1
Box 2
Box 3
Ellipse
Image
Placeholder
Button
Primary button
Link button
Gallery Page 7 Page 6 Page 5 Page 4 Page 3 Page 2 Page 1
Interactions Notes Style
Page interactions
New interaction
Add page interactions above or select a widget on the canvas to make the widget interactive
CATEGORIES
CLOTHES
SHOES
ELECTRONICS
ACCESSORIES
MAKEUP


EXP 5A
25 days left in trial
File Edit View Project Arrange Publish Team Account Help
Share
Page dimensions Auto
+ Add adaptive views
Page style Default
Page align
Fill
Visual fidelity Normal
Pages Outline
Page 1
Page 2
Page 3
Page 4
Libraries Components
All libraries
Default • Common
Box 1
Box 2
Box 3
Ellipse
Image
Placeholder
Button
Primary button
Link button
Gallery
Page 7
Page 6
Page 5
Page 4
Page 3
Page 2
Page 1
CLOTHES
T-SHIRT
RS 500
PANTS
RS 900
Interactions Notes Style
Page interactions
New interaction
Add page interactions above or select a widget on the canvas to make the widget interactive


EXP 5A
25 days left in trial
File Edit View Project Arrange Publish Team Account Help
Share
Page dimensions Auto
+ Add adaptive views
Page style Default
Page align
Fill
Visual fidelity Normal
Pages Outline
Page 1
Page 2
Page 3
Page 4
Libraries Components
All libraries
Default • Common
Box 1
Box 2
Box 3
Ellipse
Image
Placeholder
Button
Primary button
Link button
Gallery
Page 7
Page 6
Page 5
Page 4
Page 3
Page 2
Page 1
T-SHIRT
RS 500
ADD TO CART
BUY NOW
Interactions Notes Style
Page interactions
New interaction
Add page interactions above or select a widget on the canvas to make the widget interactive


EXP 5A
25 days left in trial
File Edit View Project Arrange Publish Team Account Help
Share
Page dimensions Auto
+ Add adaptive views
Page style Default
Page align
Fill
Visual fidelity Normal
Pages Outline
Page 3
Page 4
Page 5
Page 6
Libraries Components
All libraries
Default • Common
Box 1
Box 2
Box 3
Ellipse
Image
Placeholder
Button
Primary button
Link button
Gallery
Page 7
Page 6
Page 5
Page 4
Page 3
Page 2
Page 1
Interactions Notes Style
Page interactions
New interaction
Add page interactions above or select a widget on the canvas to make the widget interactive
CHECKOUT
FULL NAME:
EMAIL:
CONTACT NO :
ADDRESS:
PAY


EXP 5A
25 days left in trial
File Edit View Project Arrange Publish Team Account Help
Share
Page dimensions Auto
+ Add adaptive views
Page style Default
Page align
Fill
Visual fidelity Normal
Pages Outline
Page 2
Page 3
Page 4
Page 5
Libraries Components
All libraries
Default • Common
Box 1
Box 2
Box 3
Ellipse
Image
Placeholder
Button
Primary button
Link button
Gallery
Page 7
Page 6
Page 5
Page 4
Page 3
Page 2
Page 1
Interactions Notes Style
Page interactions
New interaction
Add page interactions above or select a widget on the canvas to make the widget interactive
CART
PROCEED TO PAYMENT
REMOVE FROM CART
PROCEED TO PAYMENT
REMOVE FROM CART
BACK



Result:

Hence, a fully interactive prototype has been developed using Axure RP.

Exercise 5b:

Simulate the life cycle stages for UI design using the RAD model and develop a small interactive interface using OpenProj

AIM:

The aim is to recreate the lifecycle stages of UI design using the RAD model and design a small interactive interface with OpenProj.

PROCEDURE:

Tool Link: https://sourceforge.net/projects/openproj/

Step 1: Requirements Planning

1. Gather Requirements:

Identify key features and functionalities needed for your interface.

with debug logs.

2. Define Use Cases:

- Specify use cases for user login and registration account.
- Create a new project.
- Add tasks
- Set durations and dependencies for each task.

Step 2: User Design

1. Sketch Initial Designs:

- Draw rough sketches of the Login and & Register screens on paper.

2. Create Digital Wireframes:

- Use a tool like Figma or Sketch to create digital wireframes.

Step 3: Rapid Prototyping

1. Develop Prototypes:

- Use a tool like Axure RP to convert wireframes into interactive prototypes.

2. Test Prototypes:

- Share prototypes with stakeholders for feedback.
- Collect feedback and iterate on the design.

Step 4: User Acceptance/Testing

1. Review Prototype:

- Conduct user and stakeholder reviews.

2. Conduct Usability Testing:

- Perform usability testing and document feedback.

Step 5: Implementation

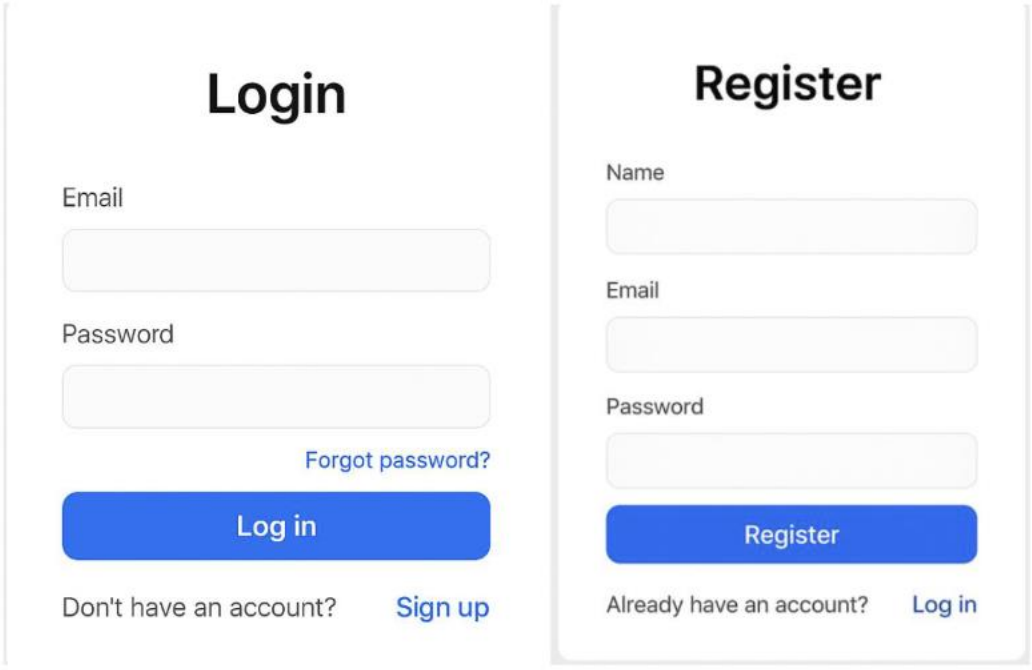
1. Develop Functional Interface:

- Implement final designs and functionalities based on feedback.

2. Integrate Backend (if required):

- Connect the UI with backend services for tasks like user authentication.

OUTPUT:



RESULT:

Demonstration of the lifecycle stages of UI design via the RAD model and development of a small interactive interface employing OpenProj has been successfully completed.

Exercise 6:

Experiment with different layouts and color schemes for an app. Collect user feedback on aesthetics and usability using GIMP(GNU Image Manipulation Program (GIMP)

Aim:

The aim is to trial different app layouts and color schemes and evaluate user feedback on aesthetics and usability using GIMP.

Procedure:

Step 1: Install GIMP

- Download from the official GIMP site and install.

Step 2: Create a New Project

- Open GIMP → File → New.
- Set dimensions (e.g., 1080x1920 for mobile).

Step 3: Design the Layout

- Use the Rectangle Select Tool to create sections (header, content, footer).
- Fill with colors using the Bucket Fill Tool.
- Add text and interactive elements with the Text and Shape/Brush Tools.
- Organize elements in layers and name them clearly (e.g., Header, Button1).

Step 4: Experiment with Colors

- Duplicate the layout (right-click image tab → Duplicate).
- Change colors using the Bucket Fill or Colorize Tool.
- Export each variant (File → Export As, e.g., Layout1.png).

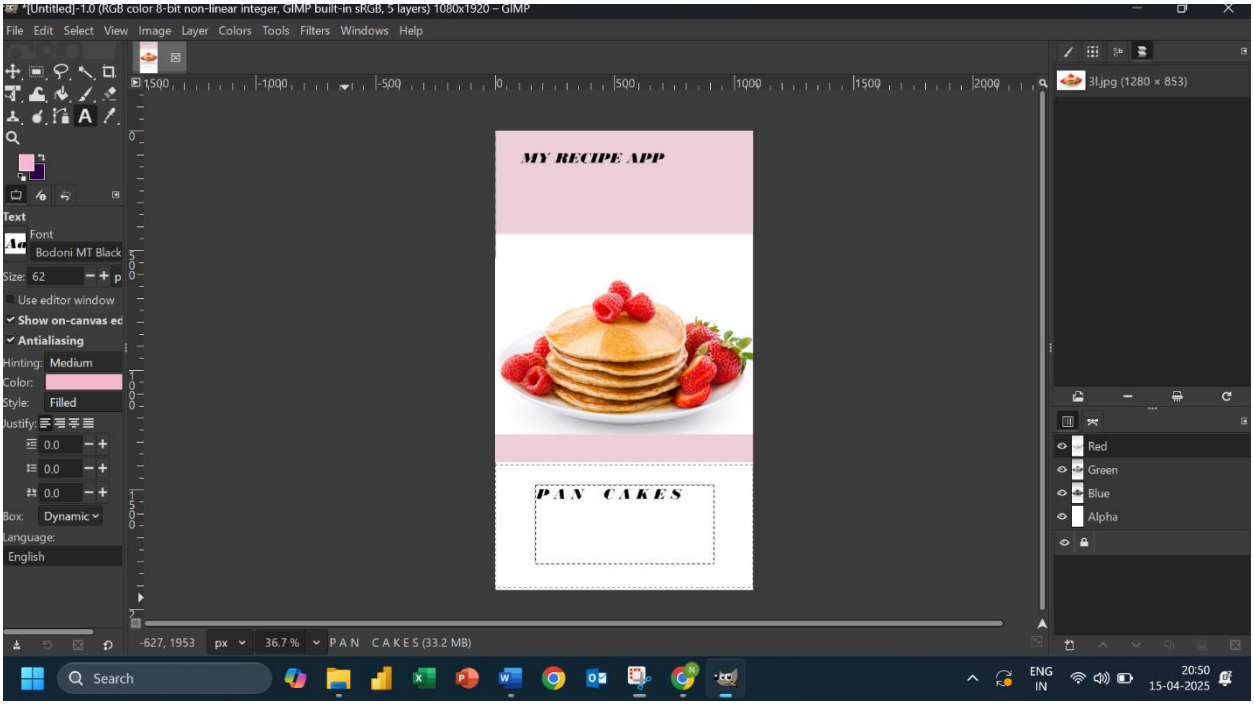
Step 5: Collect Feedback

- Create a feedback form (Google/Microsoft Forms).
- Share layout images and form with users.
- Collect and review responses.

Step 6: Refine and Finalize

- Update designs based on feedback.
- Test the final version for usability and look.

Output:



Result:

Hence, experiment with different layouts and color schemes for an app was successfully executed using GIMP.

Exercise 7a:

Develop low-fidelity paper prototypes for a banking app and convert them into digital wireframes using Pencil Project

Aim:

The aim is to develop low-fidelity paper prototypes for a banking app and convert them into digital wireframes with Pencil Project.

Procedure:

Step 1: Create Low-Fidelity Paper Prototypes

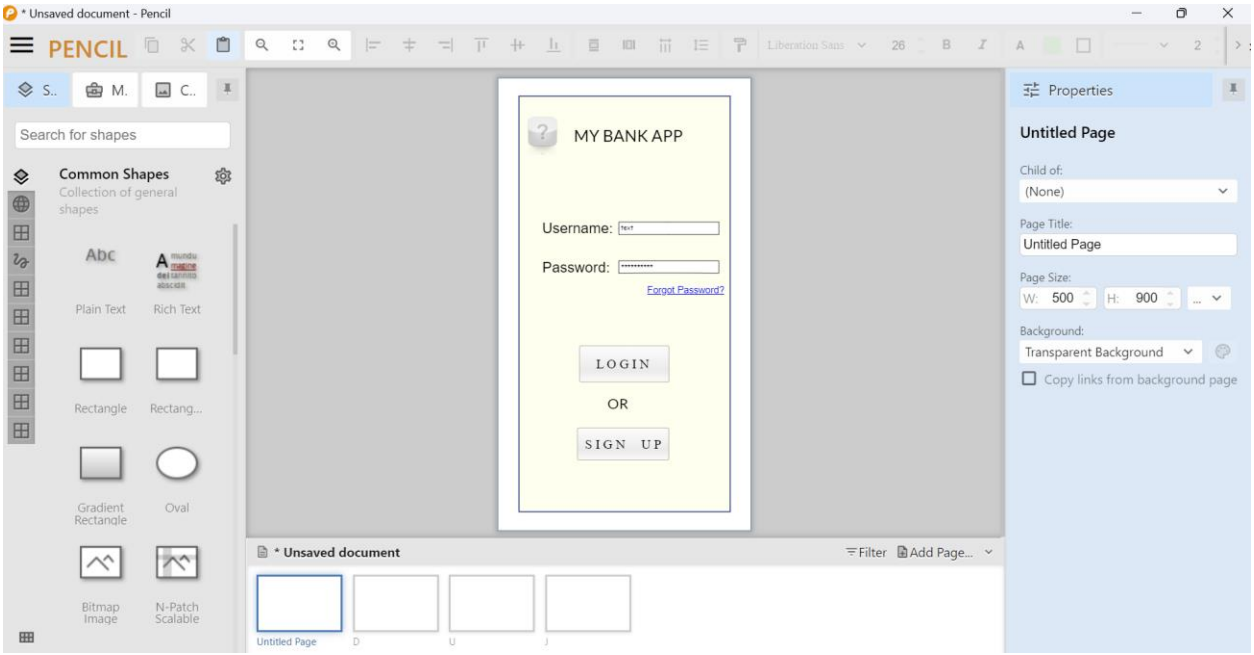
1. **Define Purpose & Features**
    - Identify core app features: login, balance check, transfers, bill pay.
2. **Sketch Layouts**
    - Use paper and pencil to draw basic screen designs with key elements (buttons, menus, forms).
3. **Iterate & Improve**
    - Gather feedback and refine sketches for better clarity and usability.

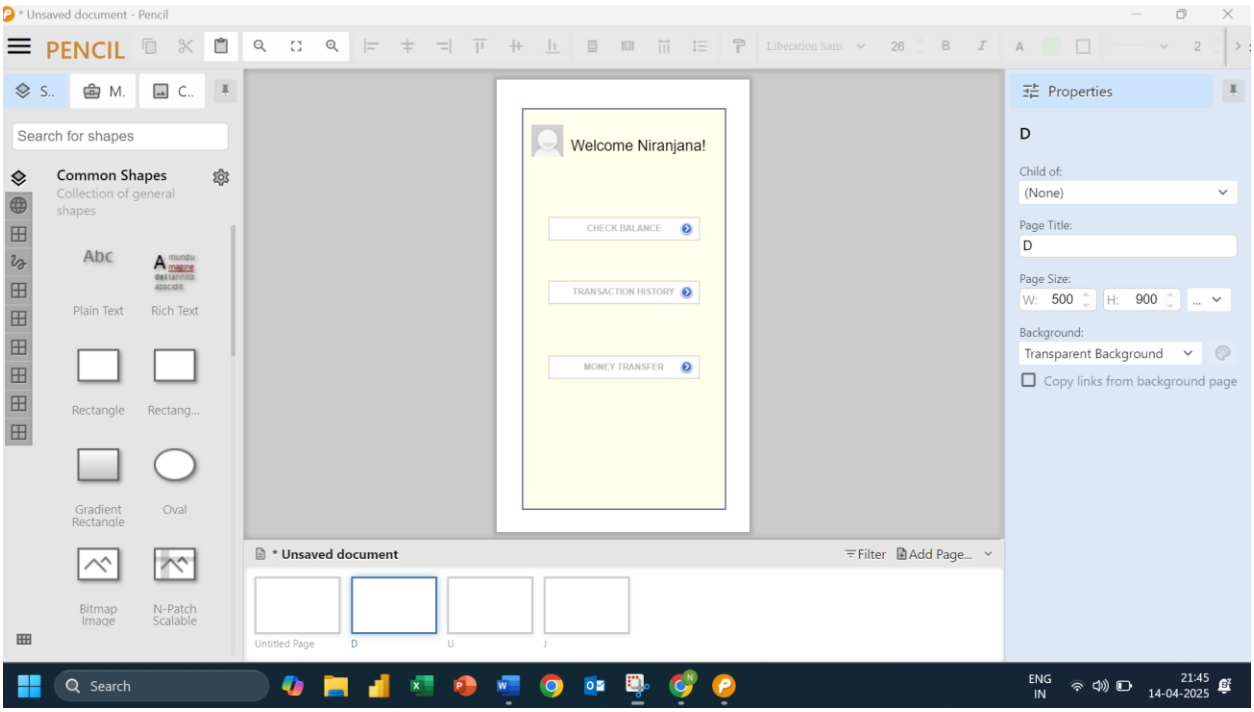
Step 2: Create Digital Wireframes with Pencil Project

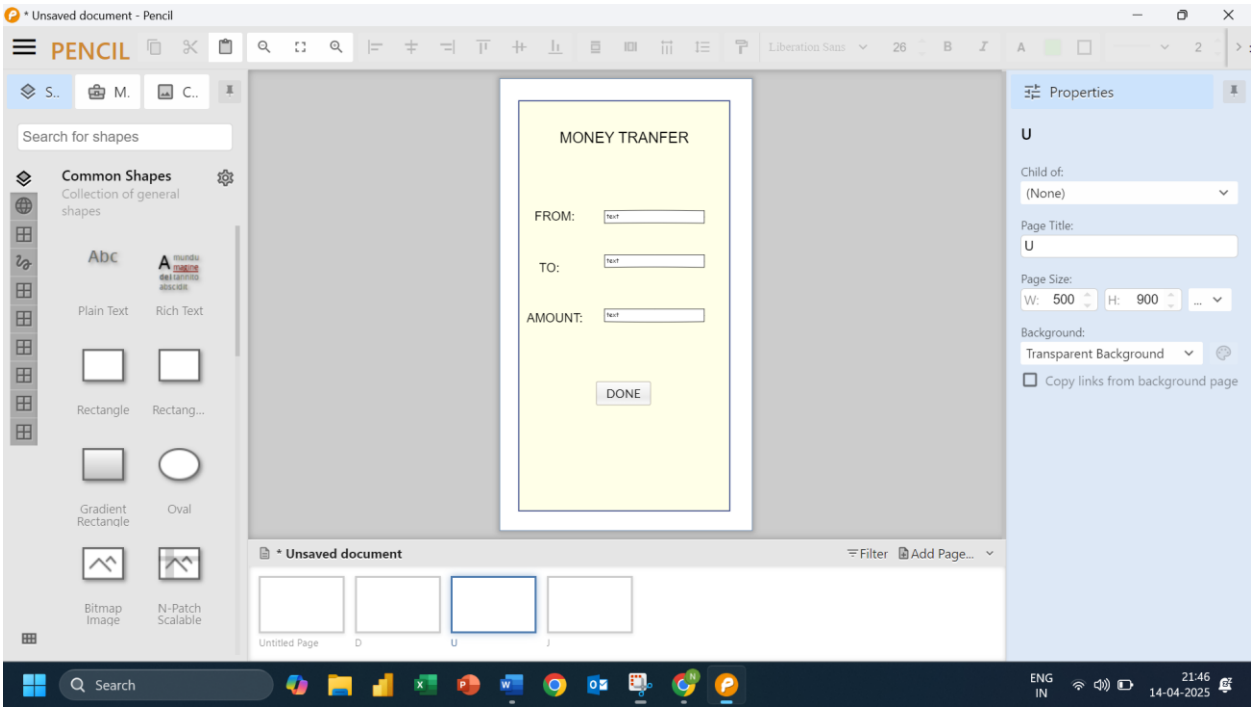
1. **Install & Set Up**
    - Download and install Pencil Project, then start a new document.
2. **Design Screens**
    - Add pages for each screen (e.g., Login, Dashboard).
    - Use built-in stencils to drag UI elements onto the canvas.
3. **Organize & Link**
    - Align elements for a clean layout.
    - Link screens to create a navigation flow.
4. **Annotate & Export**

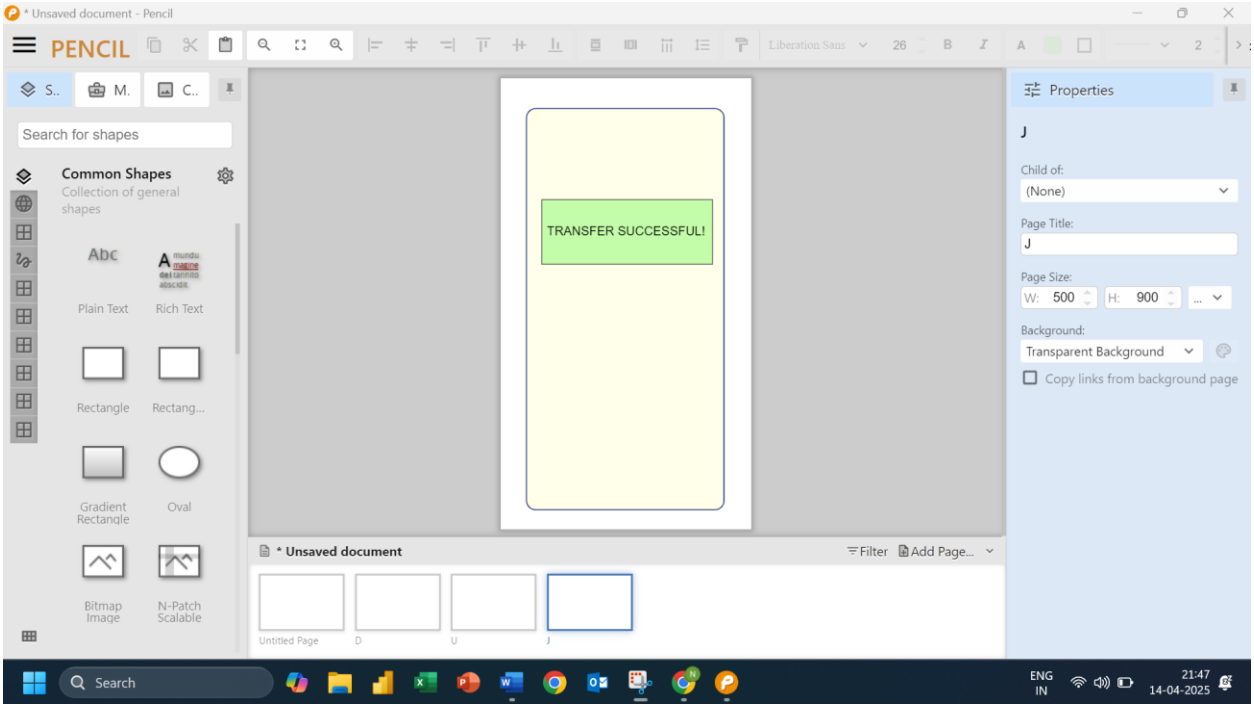
- Add notes explaining features.
- Export wireframes as PNG or PDF.

Output:









Result:

Hence, low-fidelity paper prototypes for a banking app was successfully created using Pencil Project.

Exercise 7b

Develop low-fidelity paper prototypes for a banking app and convert them into digital wireframes using Inkscape.

Aim:

The aim is to construct low-fidelity paper prototypes for a banking app and digitize them into wireframes using Inkscape.

Procedure:

Step 1: Create Low-Fidelity Paper Prototypes

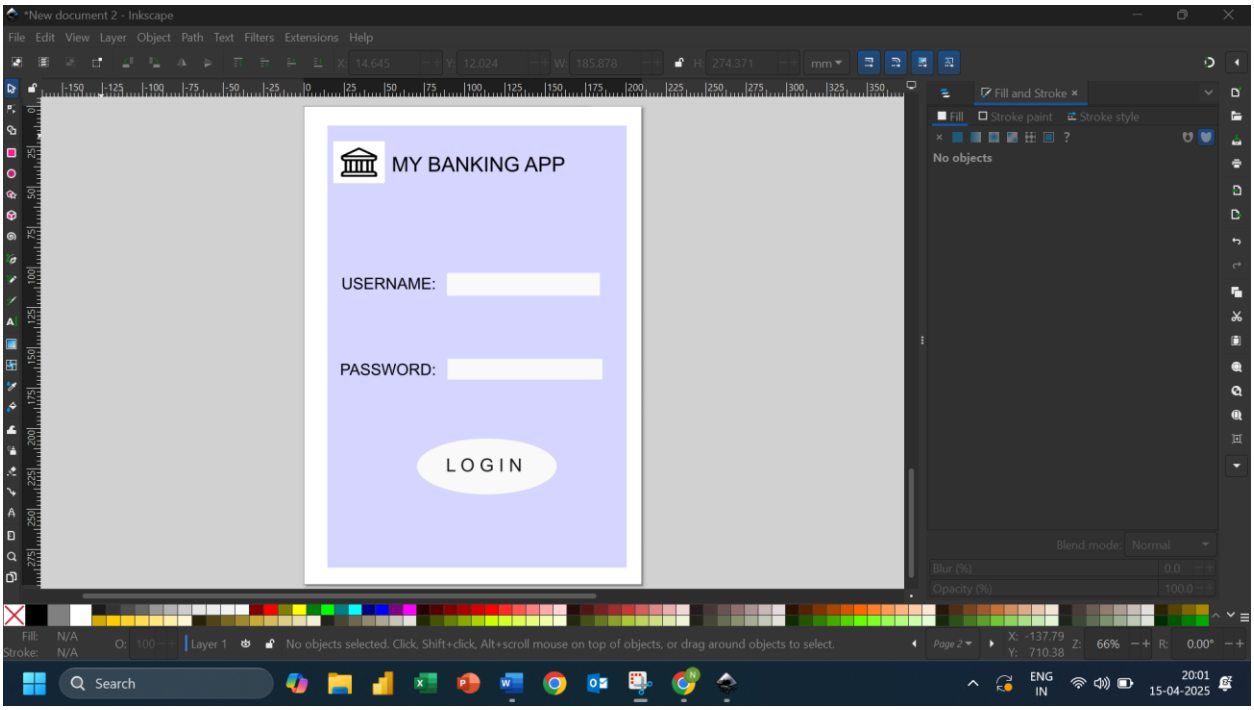
1. **Identify Core Features**
    - Define essential banking app features: login, dashboard, transfers, account management.
2. **Sketch Layouts**
    - Use paper and pencil to sketch key screens with basic UI elements (buttons, menus, inputs).
3. **Refine Designs**
    - Gather feedback and revise sketches to improve clarity and functionality.

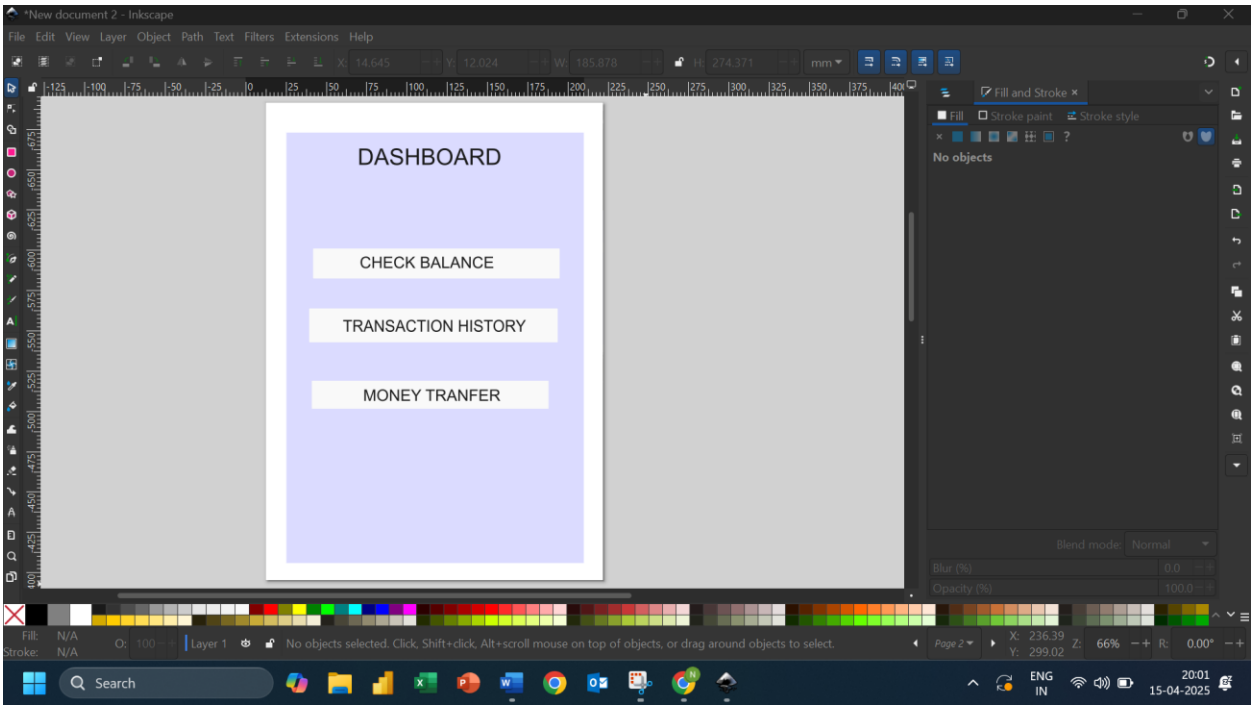
Step 2: Convert Paper Prototypes to Digital Wireframes Using Inkscape

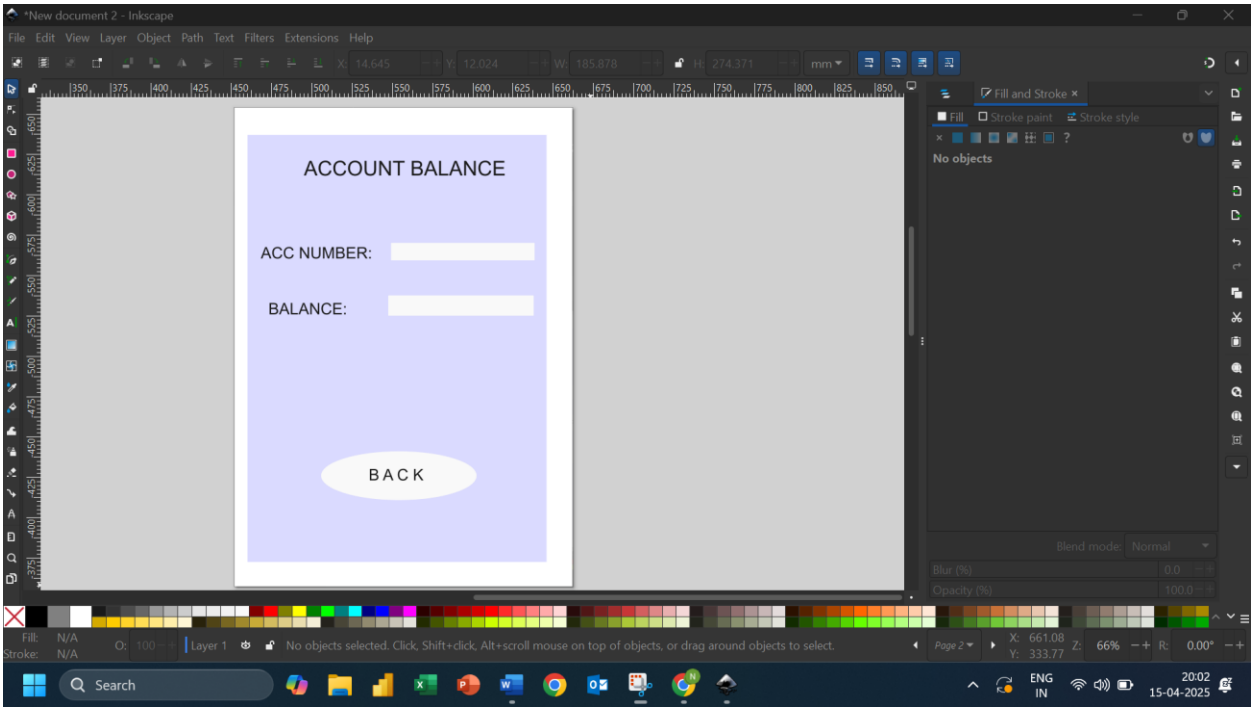
1. **Install & Set Up**
    - Download Inkscape and create a new document.
    - Set dimensions via *File > Document Properties* and enable grid (*View > Page Grid*).

2. **Design UI Elements**
    - Use shape tools for buttons, fields, icons.
    - Add text labels with the text tool.
3. **Organize Layout**
    - Align and arrange elements using alignment tools.
    - Group related elements via *Object > Group*.
4. **Create Multiple Screens**
    - Duplicate layouts for different screens (login, dashboard, etc.) using *Edit > Duplicate*.
5. **Link Screens (Optional)**
    - Add arrows or indicators to show navigation flow.
6. **Export Wireframes**
    - Export each screen as PNG via *File > Export PNG Image*.

Output:


*New document 2 - Inkscape
File Edit View Layer Object Path Text Filters Extensions Help
MY BANKING APP
USERNAME:
PASSWORD:
LOGIN
Fill and Stroke
No objects
Search
ENG
IN
20:01
15-04-2025


*New document 2 - Inkscape
File Edit View Layer Object Path Text Filters Extensions Help
DASHBOARD
CHECK BALANCE
TRANSACTION HISTORY
MONEY TRANFER
Fill and Stroke
No objects
Search
ENG
IN
20:01
15-04-2025



Result:

Hence, low-fidelity paper prototypes for a banking app were successfully developed using Inkscape.

Exercise 8a

Create storyboards to represent the user flow for a mobile app(e.g., food delivery app) using Balsamiq

Aim:

The aim is to create storyboards representing the user flow for a mobile app, such as a food delivery app, using Balsamiq.

Procedure:

1. Install Balsamiq

- Go to the official website: https://balsamiq.com/
- Download and install the Balsamiq Wireframes application suitable for your operating system (Windows, macOS, or use the web version).

2. Create a New Project

- Open Balsamiq.
- Select **"Create New Project"** from the start screen or the File menu.
- Give your project a meaningful name.

3. Add Wireframe Screens

- Click the **“+” button** (Add New Wireframe) to create a new screen.
- Repeat for each key screen that will be part of your app (e.g., Login, Home, Settings, Profile).

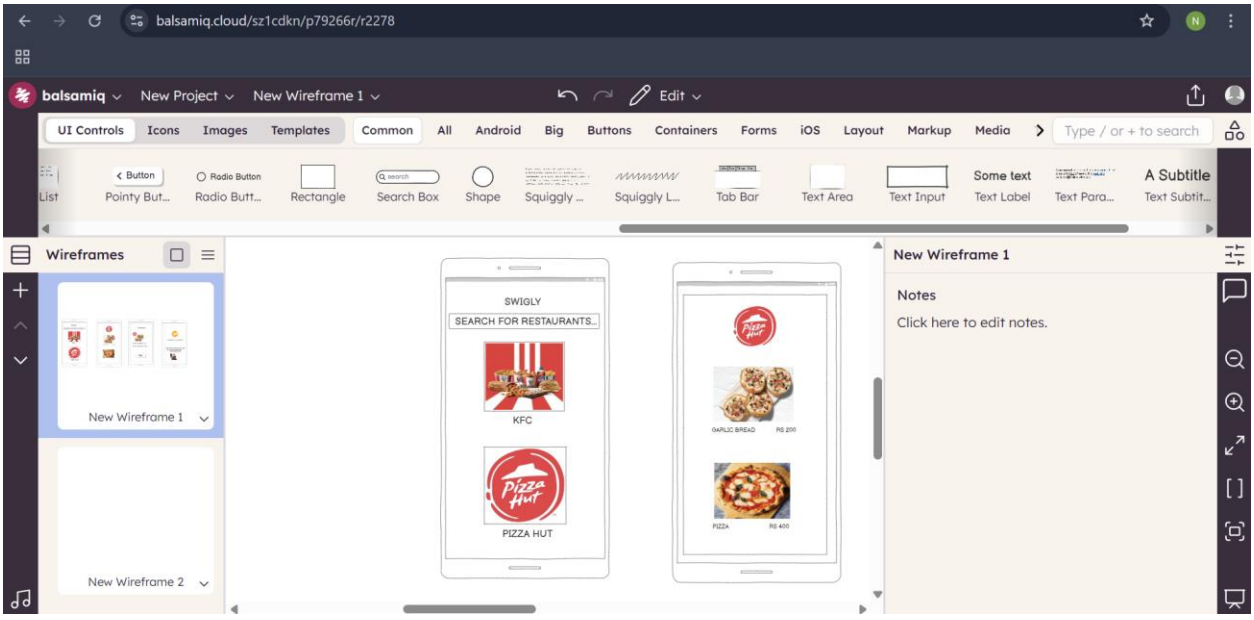
4. Design Each Screen

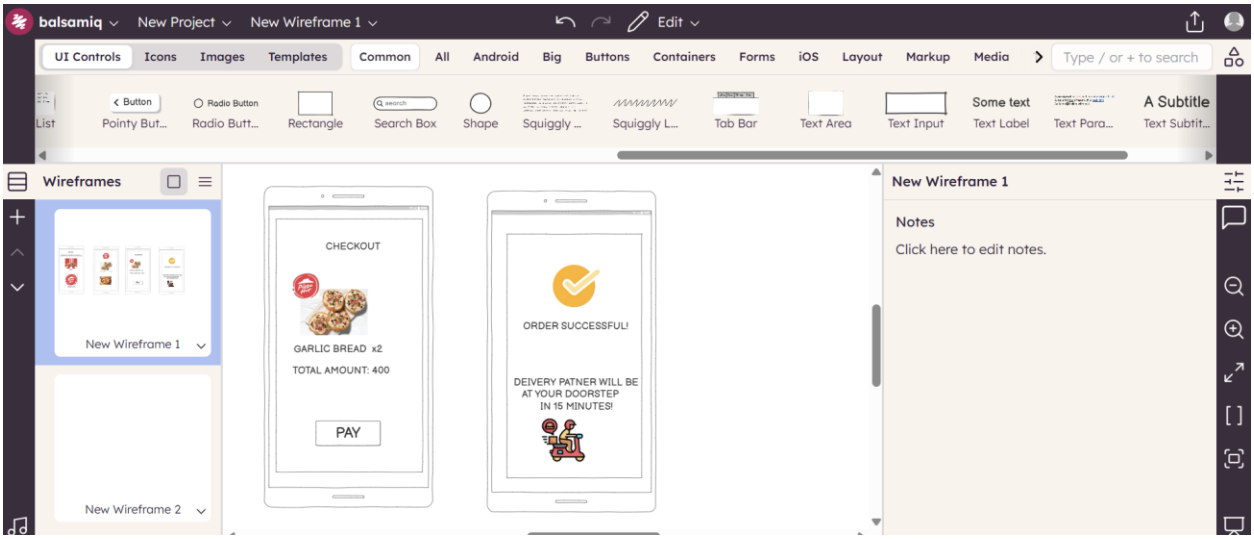
- Drag and drop UI components from the **UI Library** onto each screen.
- Include common elements such as:
    - **Buttons**
    - **Text fields**
    - **Labels**
    - **Images**
    - **Navigation bars**
- Keep the design simple and focused on layout rather than final design details.

5. Organize the Flow

- Arrange your wireframe screens in the logical order users will follow.
- Use **arrows or linking tools** to show navigation and user interactions between screens.
    - For example, a login button should link to the Home screen.

Output:


balsamiq.cloud/sz1cdkn/p79266r/r2278
balsamiq
New Project
New Wireframe 1
Edit
UI Controls
Icons
Images
Templates
Common
All
Android
Big
Buttons
Containers
Forms
iOS
Layout
Markup
Media
Type / or + to search
Wireframes
New Wireframe 1
New Wireframe 2
SWIGLY
SEARCH FOR RESTAURANTS...
KFC
PIZZA HUT
Notes
Click here to edit notes.


balsamiq
New Project
New Wireframe 1
Edit
UI Controls
Icons
Images
Templates
Common
All
Android
Big
Buttons
Containers
Forms
iOS
Layout
Markup
Media
Type / or + to search
Wireframes
New Wireframe 1
New Wireframe 2
CHECKOUT
GARLIC BREAD x2
TOTAL AMOUNT: 400
PAY
ORDER SUCCESSFUL!
DELIVERY PATNER WILL BE AT YOUR DOORSTEP IN 15 MINUTES!
Notes
Click here to edit notes.

Result:

Storyboards created using Balsamiq clearly represent the user flow of a food delivery app, showcasing all key screens and interactions from searching restaurants to order confirmation.

Exercise 8b

Create storyboards to represent the user flow for a mobile app (e.g., food delivery app) using OpenBoard

Aim:

To map out the user flow for a mobile app (e.g., a food delivery app), storyboards will be designed using OpenBoard.

Procedure:

1. Install OpenBoard

- Visit the official OpenBoard website: https://openboard.ch/
- Download and install the version compatible with your operating system (Windows, macOS, or Linux).

2. Create a New Document

- Launch OpenBoard.
- Click **"New"** or select **"File > New"** to create a blank document for your storyboard project.

3. Add Frames for Each Screen

- Use the **drawing area** to create individual frames representing each key screen of your app.

4. Sketch Each Screen

- Use the **Pen** or **Shape tools** (rectangle, ellipse, line, etc.) to sketch out the UI of each screen.
- Focus on key UI components such as:
    - **Buttons**
    - **Text fields**
    - **Icons**

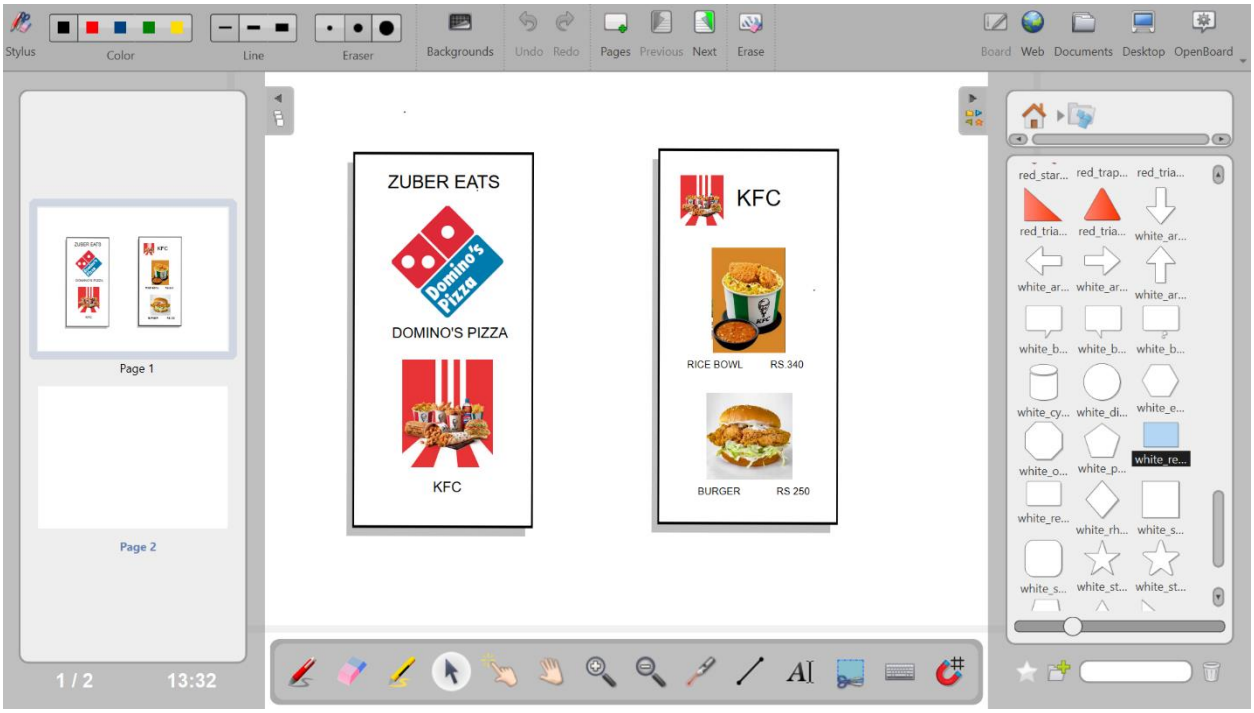
- **Images**

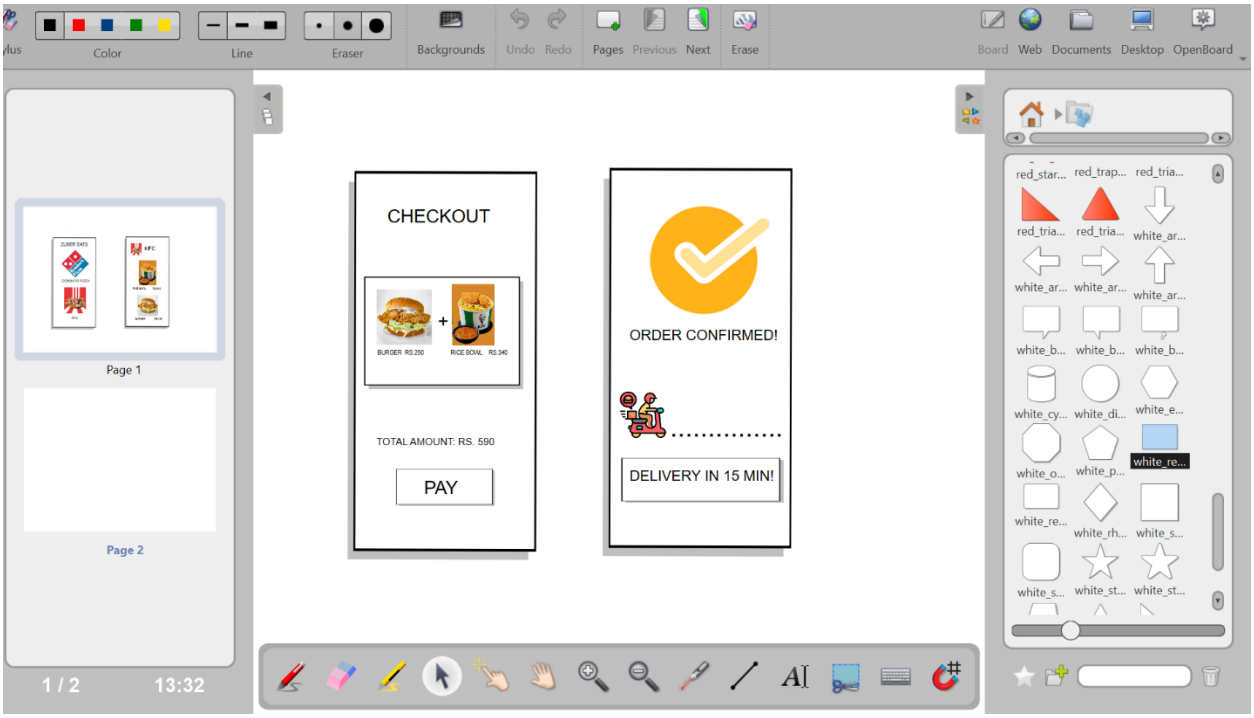
- Keep sketches simple and clear—focus on layout, not visual polish.

5. Organize the Flow

- Arrange the frames in the order of the **user journey** (e.g., from login to dashboard to settings).
- Use **arrows or connector lines** to indicate navigation paths and user actions between screens.

Output:





Result:

Storyboards for the mobile food delivery app were created in OpenBoard, clearly representing the user journey from browsing to order confirmation.

Exercise 9:

Design input forms that validate data (e.g., email, phone number) and display error messages using HTML/CSS,JavaScript (with Validator.js)

Aim:

The aim is to design input forms that validate data, such as email and phone number, and display error messages using HTML/CSS and JavaScript with Validator.js.

Procedure:

Step 1: Setting Up the HTML Form

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Form Validation</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <form id="myForm">
      <label for="email">Email:</label>
      <input type="email" id="email" name="email" required>
```

```
    <span id="emailError" class="error"></span>

    <label for="phone">Phone Number:</label>
    <input type="text" id="phone" name="phone" required>
    <span id="phoneError" class="error"></span>

    <button type="submit">Submit</button>
   </form>
  </div>

  <script
src="https://cdnjs.cloudflare.com/ajax/libs/validator/13.6.0/validator.min.js"></scri
pt>
  <script src="script.js"></script>
</body>
</html>
```

Step 2: Styling the Form with CSS

```
/* style.css */
body {
  font-family: Arial, sans-serif;
  background-color: #f4f4f4;
  display: flex;
  justify-content: center;
  align-items: center;
```

```
  height: 100vh;
  margin: 0;
}

.container {
  background-color: white;
  padding: 20px;
  border-radius: 5px;
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
}

form {
  display: flex;
  flex-direction: column;
}

label {
  margin-bottom: 5px;
}

input {
  margin-bottom: 10px;
  padding: 10px;
  border: 1px solid #ccc;
```

```
  border-radius: 3px;
}

button {
  padding: 10px;
  background-color: #28a745;
  color: white;
  border: none;
  border-radius: 3px;
  cursor: pointer;
}

button:hover {
  background-color: #218838;
}

.error {
  color: red;
  font-size: 0.875em;
}
```

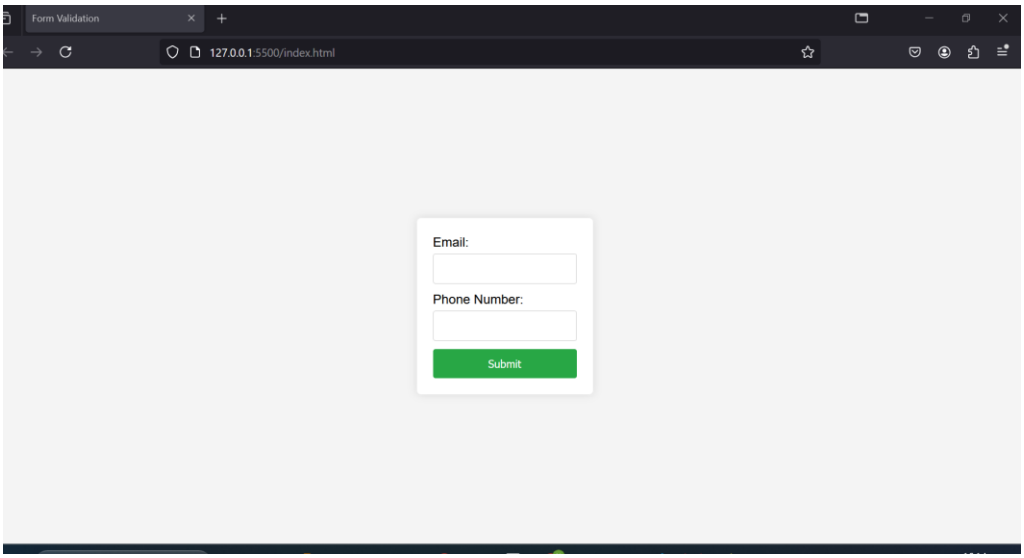
Step 3: Adding JavaScript for Validation

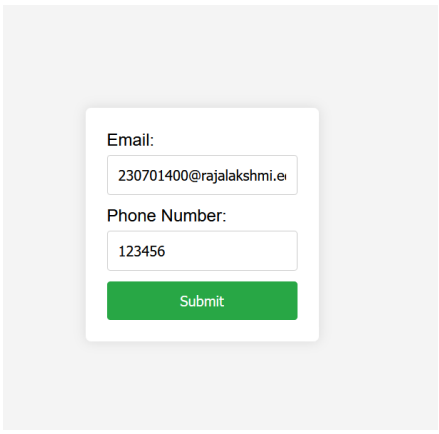
```
/* script.js */
document.getElementById('myForm').addEventListener('submit', function (e) {
```

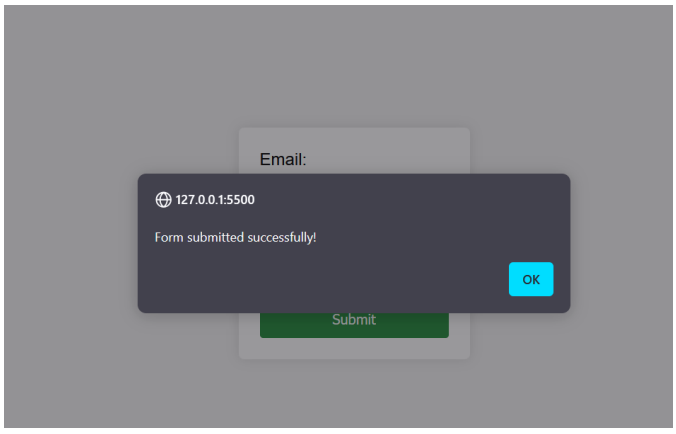
```
  e.preventDefault();

  let email = document.getElementById('email').value;
  let phone = document.getElementById('phone').value;

  let emailError = document.getElementById('emailError');
  let phoneError = document.getElementById('phoneError');

  // Clear previous error messages
  emailError.textContent = '';
  phoneError.textContent = '';

  // Validate email
  if (!validator.isEmail(email)) {
    emailError.textContent = 'Please enter a valid email address.';
  }

  // Validate phone number
  if (!validator.isMobilePhone(phone, 'any')) {
    phoneError.textContent = 'Please enter a valid phone number.';
  }

  // If no errors, submit the form (for demonstration purposes, we'll just log the values)
  if (validator.isEmail(email) && validator.isMobilePhone(phone, 'any')) {
```

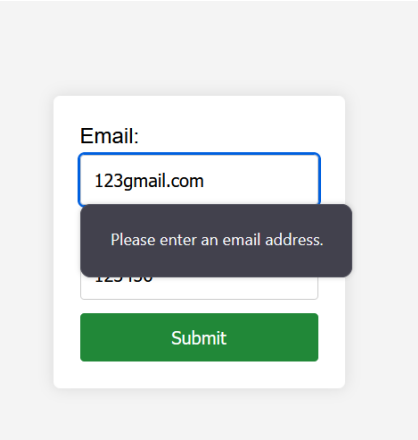
```
    console.log('Email:', email);
    console.log('Phone:', phone);
  }
});
```

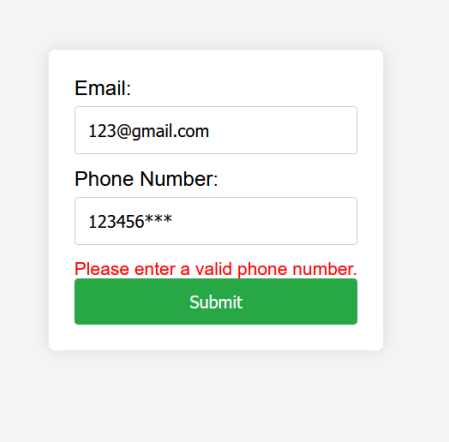
Output:











Result:

The input form was successfully designed using HTML and styled with CSS. Validation of user inputs such as email and phone number were effectively implemented using JavaScript with the Validator.js library. The form displayed appropriate error messages for invalid inputs and allowed submission only when all data was valid, thereby ensuring accurate and user-friendly data entry.

Exercise 10:

Create a data visualization (e.g., pie charts, bar graphs) for an inventory management system using javascript.

Aim:

The aim is to create data visualizations, such as pie charts and bar graphs, for an inventory management system using JavaScript.

Procedure:

Step 1: Set Up Your HTML File

1. Create a file and name it index.html.
2. Add a <canvas> element for the Pie Chart and Bar Chart.
3. Include the Chart.js library and link your JavaScript file (script.js).

Code (index.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Inventory Management Visualization</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      text-align: center;
      margin: 50px;
```

```
    }
    canvas {
      margin: 20px auto;
    }
  </style>
</head>
<body>
  <h1>Inventory Management System</h1>
  <canvas id="pieChart" width="400" height="400"></canvas>
  <canvas id="barChart" width="400" height="400"></canvas>

  <!-- Chart.js Library -->
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
  <!-- Link to JavaScript File -->
  <script src="script.js"></script>
</body>
</html>
```

Step 2: Create the JavaScript File for Charts

1. Create a new file named script.js.
2. Add the inventory data.
3. Create a **Pie Chart** for category distribution.
4. Create a **Bar Chart** for items in stock.

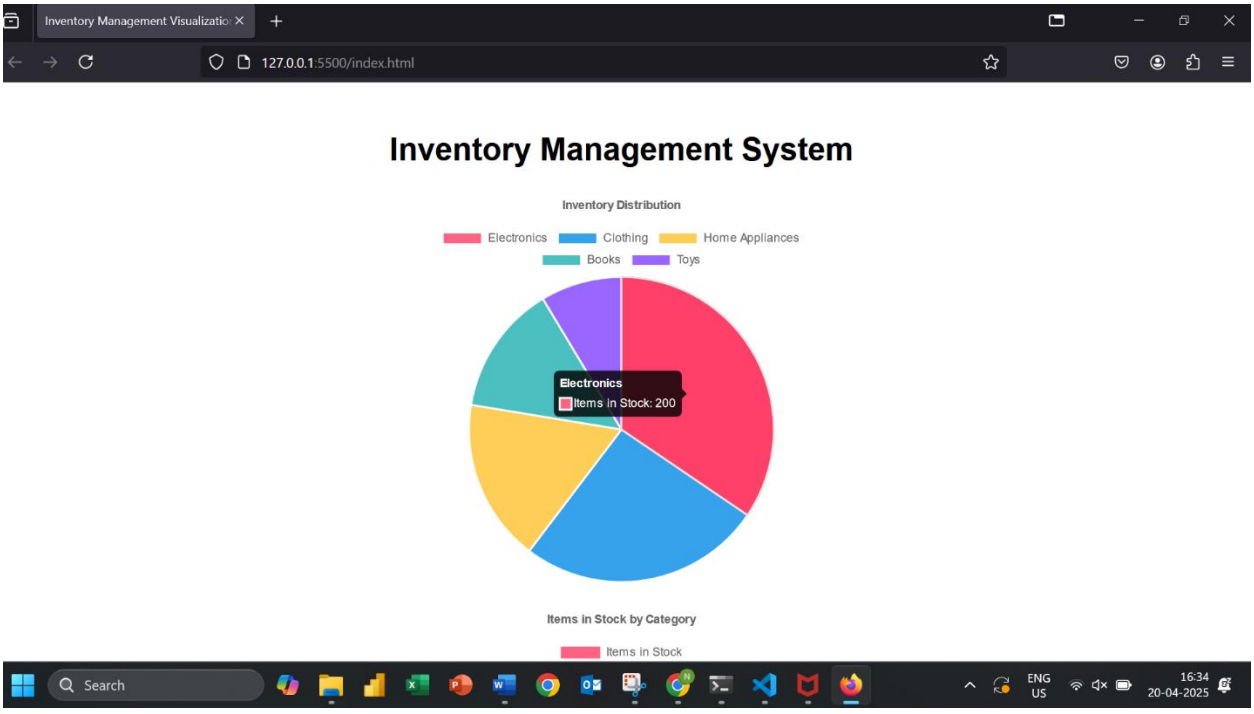
Code (script.js)

```
// Data for the inventory
const inventoryData = {
  labels: ['Electronics', 'Clothing', 'Home Appliances', 'Books', 'Toys'],
  datasets: [
    {
      label: 'Items in Stock',
      data: [200, 150, 100, 80, 50],
      backgroundColor: [
        '#FF6384',
        '#36A2EB',
        '#FFCE56',
        '#4BC0C0',
        '#9966FF'
      ]
    }
  ]
};

// Creating the Pie Chart
const ctxPie = document.getElementById('pieChart').getContext('2d');
const pieChart = new Chart(ctxPie, {
  type: 'pie',
  data: inventoryData,
```

```
  options: {
    responsive: true,
    plugins: {
      title: {
        display: true,
        text: 'Inventory Distribution'
      }
    }
  }
});

// Creating the Bar Chart
const ctxBar = document.getElementById('barChart').getContext('2d');
const barChart = new Chart(ctxBar, {
  type: 'bar',
  data: inventoryData,
  options: {
    responsive: true,
    plugins: {
      title: {
        display: true,
        text: 'Items in Stock by Category'
      }
    },
```

```
    scales: {
      y: {
        beginAtZero: true
      }
    }
  }
});
```

Output:



Result:

Successfully created data visualizations (Pie and Bar charts) using Chart.js to represent the stock levels in different inventory categories for an inventory management system.